



UrbanFlow

Dossier projet, Titre professionnel CDSD (RNCP 36146)

Réponse à l'appel d'offres de Nantes Métropole

Session septembre 2026

COUSSOT, Elwen, B3DEV, Concepteur Développeur de Solutions Digitales

Dépôt GitHub : github.com/iMxSquash/urbanflow

Maquettes Figma : figma.com/design/.../Urbanflow

Sommaire

1. Contexte et objectifs	3
1.1 Reformulation du besoin client	3
1.2 Parties prenantes	3
1.3 Étude concurrentielle	5
1.4 Personas	5
1.5 KPIs de succès	6
2. Pilotage, analyse et gestion du projet	7
2.1 Comparatif des solutions techniques	7
2.2 Justification des choix techniques	8
2.3 Méthodologie	9
2.4 Amélioration continue	10
2.5 Planning et charge	10
2.6 Budget et chiffrage	11
3. Conception : diagrammes UML	13
3.1 Cas d'utilisation	13
3.2 Séquences	14
3.3 Communication	14
4. Spécifications du planificateur multimodal	16
4.1 Fonctionnelles	16
4.2 Techniques	17
4.3 Cas limites	20
4.4 Boutique de récompenses	21
4.5 Traçabilité	21
5. Architecture technique	22
5.1 Architecture globale	22
5.2 Monolithe modulaire	23
5.3 Pattern Stratégie	23
5.4 Maintenabilité	23
5.5 Interopérabilité	24
5.6 Modèle de données	24
6. Contraintes transversales	27
Tableau de synthèse	27
6.1 PWA installable	27
6.2 Responsive et expérience utilisateur	28
6.3 Normes et standards	28
6.4 Sécurité OWASP	28
6.5 Éco-conception	29
6.6 Géolocalisation	29
6.7 Accessibilité WCAG 2.1 AA	29
6.8 RGPD	29
6.9 Interopérabilité	30
6.10 Performances mobilité	30
6.11 Sécurité des données de déplacement	30
6.12 Normes d'accessibilité transport	31
7. Qualité, tests et gestion des bogues	32
7.1 Stratégie de tests	32
7.2 Workflow bogues	33
7.3 Philosophie de correction	33
7.4 Mode démo	33
8. Risques, perspectives et conclusion	35
8.1 Registre de risques	35
8.2 Perspectives	37
8.3 Conclusion	37
Annexe A : demande initiale du client	38

1. Contexte et objectifs

1.1 Reformulation du besoin client

Nantes Métropole, collectivité de 500 000 habitants portant une politique active de transition écologique, souhaite réduire la dépendance à l'automobile en offrant à ses citoyens une alternative numérique unifiée, accessible et engageante. L'email de Claire Hénette liste neuf fonctionnalités. L'équipe les a classées selon leur faisabilité dans le périmètre d'un MVP solo livrable en 12 semaines.

Fonctionnalités retenues au MVP (périmètre proposé)

Fonctionnalité demandée	Interprétation produit
Planificateur multimodal intelligent	Calcul d'itinéraires combinant bus, tramway, train, vélo, marche, scooter électrique, navibus
"IA" d'optimisation (trafic, météo, préférences)	Moteur d'optimisation multicritères déterministe (durée, CO2, confort) pondéré par profil utilisateur. La météo temps réel (OpenWeatherMap, cache 10 min) est intégrée dans le score de confort. Ce choix est justifié dans la section Pilotage.
Gamification avec récompenses	Points par trajet éco-responsable, badges, boutique de récompenses échangeables auprès de partenaires locaux (Bicloo, Naolib, sites culturels nantais)
Calcul et suivi empreinte carbone	Score CO2 par trajet, facteurs ADEME, économie vs voiture
Tableau de bord citoyen	Historique de trajets, statistiques de mobilité, progression gamification

Fonctionnalités reportées en V2 (hors périmètre MVP)

Fonctionnalité	Raison du report
Réservation unifiée (vélos, parkings, covoiturage)	Nécessite des contrats partenaires et des APIs de paiement : périmètre commercial, hors MVP académique
Plateforme de covoiturage dynamique	Fonctionnalité de mise en relation : complexité RGPD et sécurité supplémentaire, périmètre distinct
Alertes/notifications temps réel (SIRI)	L'interface SIRI-Lite Naolib est identifiée et documentée ; l'intégration complète est planifiée en V2
Signalement collaboratif voirie	Fonctionnalité civique à fort potentiel mais sans impact sur le MVP transport ; périmètre isolé

Besoins implicites identifiés, non formulés dans l'email

Trois besoins transversaux émergent de l'analyse du contexte, sans figurer explicitement dans la demande.

Fiabilité en mobilité contrainte. Un citoyen en déplacement dispose d'une connexion réseau variable. Le cas type : l'utilisateur planifie son trajet en ligne, puis perd le réseau dans un tunnel de tramway et doit pouvoir reconsulter les étapes et la carte de l'itinéraire déjà calculé. La solution répond par une stratégie hors ligne côté client (service worker pour l'app shell et les tuiles consultées, persistance des derniers itinéraires et de l'état utilisateur en IndexedDB). Le calcul d'un nouvel itinéraire reste dépendant des APIs de routage : il est hors périmètre hors ligne en MVP, et documenté comme tel. Cette stratégie est distincte du mode démo interne (variable `DEMO_MODE=true`, bascule le backend sur des fichiers JSON statiques), qui est un filet de sécurité de développement et de soutenance, jamais activé pour l'utilisateur final.

Confiance dans le traitement des données GPS. Les données de déplacement sont des données personnelles sensibles. L'utilisateur doit consentir explicitement à la géolocalisation et disposer d'un droit d'effacement effectif. Ce besoin implicite conditionne l'adoption.

Inclusion : PMR et fracture numérique. Nantes Métropole porte une politique d'accessibilité universelle. La solution doit filtrer les itinéraires selon les capacités de l'utilisateur (mobilité réduite) et rester utilisable sans smartphone haut de gamme (bundle léger, chargement rapide sur réseau 3G).

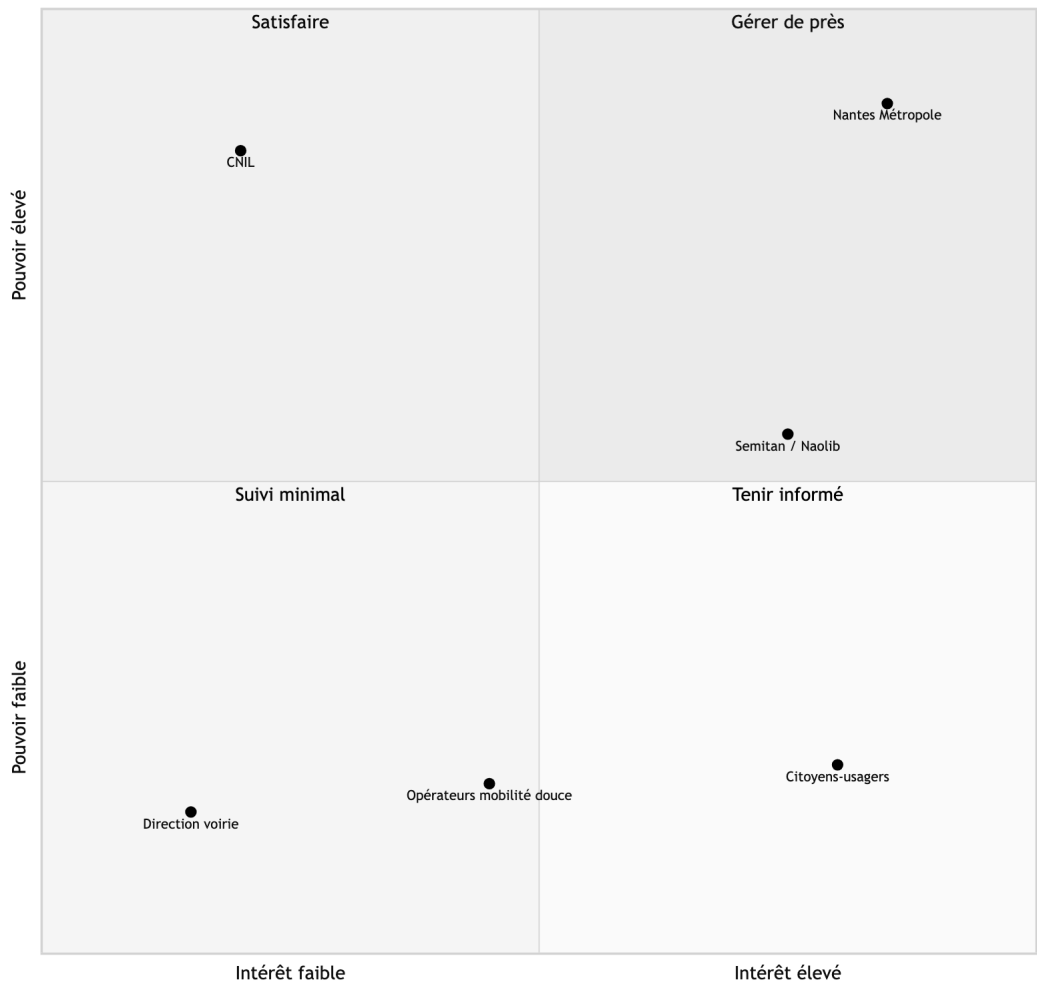
1.2 Parties prenantes

La matrice ci-dessous classe les acteurs selon leur pouvoir de décision sur le projet et leur intérêt dans son succès.

Acteur	Pouvoir	Intérêt	Nature de l'engagement
Nantes Métropole (Claire Hénette)	Élevé	Élevé	Commanditaire, financeur, valideur des livrables
Citoyens-usagers	Faible	Élevé	Utilisateurs finaux : leur adoption est le KPI politique central
Semitan/Naolib	Moyen	Élevé	Opérateur du réseau Naolib, fournisseur de données TC (GTFS, SIRI)

Acteur	Pouvoir	Intérêt	Nature de l'engagement
Opérateurs mobilité douce (Bicloo, trottinettes)	Faible	Moyen	Fournisseurs de données GBFS (General Bikeshare Feed Specification) ; aucun contrat direct au MVP
Direction voirie Nantes Métropole	Faible	Faible	Destinataire des signalements collaboratifs V2 ; sans impact sur le MVP
CNIL	Élevé	Faible	Autorité de contrôle RGPD ; pouvoir sanction élevé, engagement indirect (conformité à maintenir)

Matrice pouvoir / intérêt



Matrice pouvoir/intérêt

La matrice positionne les six acteurs identifiés en A2 selon les mêmes valeurs Pouvoir/Intérêt que le tableau de la section Contexte, répartis en quatre zones d'action.

Gérer de près (pouvoir élevé, intérêt élevé). Nantes Métropole est commanditaire, financeur et valideur des livrables : elle exige une communication rapprochée et continue tout au long du projet. Semitan/Naolib se situe à la frontière haute de cette zone : opérateur du réseau et fournisseur des données GTFS/SIRI, sans son accord opérationnel l'intégration temps réel prévue en V2 ne peut pas avancer.

Satisfaire (pouvoir élevé, intérêt faible). La CNIL n'intervient pas dans les choix produit au quotidien, mais dispose d'un pouvoir de sanction élevé sur tout traitement de données de géolocalisation non conforme. La conformité RGPD doit rester irréprochable sans qu'un dialogue actif soit nécessaire.

Tenir informé (pouvoir faible, intérêt élevé). Les citoyens-usagers n'ont aucun levier de décision sur le projet, mais leur adoption est le KPI politique central de la collectivité : leurs retours d'usage doivent remonter même sans pouvoir formel de leur part.

Suivi minimal (pouvoir faible, intérêt faible). La Direction voirie de Nantes Métropole, destinataire potentiel du signalement collaboratif reporté en V2, n'a pas d'impact sur le périmètre MVP actuel. Les opérateurs de mobilité douce (Bicloo, trottinettes) se situent à la frontière de cette zone : simples fournisseurs de données GBFS consommées en lecture seule, sans contrat direct au MVP.

Acteur décisif pour la réussite opérationnelle : Semitan/Naolib. L'accès aux données temps réel (prochains passages, perturbations) conditionne la qualité perçue de la solution. Le MVP s'appuie sur les données ouvertes disponibles (GTFS statique via

1.3 Étude concurrentielle

Quatre solutions sont présentes sur le marché ou utilisées par les citoyens nantais. Le tableau suivant les positionne face à UrbanFlow.

Solution	Forces	Limites	Différence UrbanFlow
Google Maps	Couverture mondiale, notoriété, temps réel	Données revendues à des tiers, pas de scoring CO2 sur facteurs ADEME locaux, pas de gamification, carte bancaire pour les APIs	Souveraineté des données, ancrage territorial Nantes, éco-scoring sourcé ADEME, gamification
Citymapper	UX multimodale soignée, mis à jour en temps réel, intègre les vélos en libre-service dans les villes couvertes	Ne couvre pas Nantes, disponible dans un nombre restreint de métropoles mondiales ; modèle propriétaire fermé ; pas de scoring CO2 sourcé sur des facteurs ADEME	Couverture native du territoire nantais, open data, sans dépendance à un éditeur tiers
Moovit	Base de données TC très riche, couverture mondiale, contributions communautaires, propose aussi des itinéraires vélo et marche dans plusieurs villes	Pas de calcul d'empreinte carbone sourcé sur des facteurs ADEME locaux, pas de gamification, modèle de données propriétaire fermé	Éco-scoring ADEME, gamification incitative, ancrage territorial Nantes
Application officielle Naolib (Semitan)	Source de données officielle, en temps réel sur le réseau Naolib	Mono-mode (transports en commun uniquement), pas multimodal, pas d'éco-scoring	Multimodalité complète, score CO2, profil de préférences utilisateur

Proposition de valeur unique. Aucune solution existante ne combine les quatre axes suivants sur le territoire nantais : multimodalité complète (TC + modes actifs + Bicloo), éco-scoring sourcé sur les facteurs ADEME, gamification incitative à la mobilité durable, et souveraineté des données (hébergement européen, données GPS non revendues).

1.4 Personas

Deux personas représentent la cible de cœur du produit. Ils sont construits depuis l'analyse du besoin client et du contexte territorial nantais. La question d'accessibilité PMR (personnes à mobilité réduite), transversale à l'ensemble de la solution, est traitée dans la section dédiée à l'accessibilité plutôt qu'en persona distinct : elle conditionne des choix de conception, pas un segment d'acquisition.

Léa, 27 ans, cible primaire

Profil. Chargée de communication, quartier Doulon (est de Nantes), bureau sur l'île de Nantes. Pendulaire quotidienne, vélo en été, tramway ligne 1 quand il pleut. À l'aise avec les apps mobiles.

Job-to-be-done. Rejoindre son bureau à l'heure sans avoir à arbitrer chaque matin entre ses options de transport : l'information doit venir à elle, pas l'inverse.

Points de friction (par sévérité) 1. Elle ignore si le vélo est pertinent ce matin-là : météo, vent, heure de pointe. Résultat : elle prend la décision par défaut (tram) même quand le vélo serait plus rapide. Sévère, quotidien. 2. Comparer tramway, vélo et Bicloo exige de jongler entre trois apps différentes. Friction quotidienne à chaque départ. 3. Elle ne voit pas l'impact concret de ses choix de mobilité et ne sait pas si ses efforts comptent. Faible, mais frein à la fidélité long terme.

Gains attendus 1. Une recommandation contextualisée (météo + heure) en moins de 10 secondes, sans saisie complexe. 2. La certitude d'arriver à l'heure : aucun retard lié à un mauvais choix de mode. 3. Une progression visible de son impact CO2, consultable quand elle le souhaite.

Insight inattendu. Léa n'est pas motivée par l'écologie en premier lieu. Elle adopte la mobilité douce par confort et gain de temps, pas par conviction environnementale : l'éco-scoring et la gamification ne sont pas ce qui la fait basculer vers un nouveau mode de transport, elle l'a déjà fait. Leur rôle produit n'est donc pas la conversion mais la rétention : ce qui la fait revenir consulter l'application une fois son trajet du jour trouvé, c'est la visibilité de ce qu'elle accomplit déjà. Conséquence produit : le score CO2 et la progression de gamification restent présents mais jamais en avant-plan du parcours de calcul d'itinéraire ; ils trouvent leur place dans le tableau de bord, consulté par choix et non imposé à chaque recherche.

Adéquation produit. Le planificateur multimodal avec profil "rapide" et météo intégrée répond directement à son job de rejoindre son bureau à l'heure. La gamification (points, badges, boutique de récompenses) et le tableau de bord de mobilité tiennent leur place au périmètre MVP grâce à ce même persona : sans mécanisme de visibilité de la progression, rien ne la ramène vers l'application une fois l'itinéraire du jour trouvé. C'est Léa qui porte, en rétention plutôt qu'en acquisition, les fonctionnalités qui différencient le MVP au-delà du planificateur multimodal seul. Friction principale identifiée : si le temps de calcul dépasse 2-3 secondes le matin, elle quitte l'app.

Marc, 54 ans, cible secondaire

Profil. Technicien de maintenance, Vertou (commune limitrophe au sud de Nantes). Vient en voiture jusqu’au parking de la station de tramway Vertou chaque matin, prend le tram jusqu’au centre. N’a jamais envisagé d’alternative à cette routine.

Job-to-be-done. Maintenir la fiabilité de son trajet domicile-travail sans effort supplémentaire, tout en répondant à la pression économique et sociale de réduire l’usage de la voiture.

Points de friction (par sévérité) 1. Il ne fait pas confiance aux apps de transport : “ça ne connaît pas mon trajet”. Barrière psychologique d’entrée, sévère. 2. Il n’a pas imaginé le park-and-ride comme alternative structurée : pour lui, soit c’est voiture complète, soit c’est tram depuis Vertou centre. La combinaison n’existe pas dans sa représentation mentale. Sévère. 3. Il ne dispose d’aucun argument économique chiffré pour justifier un changement d’habitude. Sans ce chiffre, l’effort perçu dépasse le bénéfice perçu. Moyen.

Gains attendus 1. Un itinéraire qui ne soit pas plus compliqué que sa routine actuelle : une correspondance maximum. 2. Un calcul d’économie mensuelle concret (coût voiture vs TC + parking P+R). 3. La preuve que l’alternative est fiable : aucune mauvaise surprise non anticipée.

Insight inattendu. Marc n’a pas besoin d’être convaincu par l’écologie. L’argument CO2 ne l’atteint pas en point d’entrée et peut même générer un rejet (“encore un truc pour les bobos”). Ce qui le convainc, c’est la simplicité du parcours et l’argument économique chiffré. Conséquence produit : pour ce profil, le score CO2 apparaît en second plan, après la durée et l’économie financière. L’interface doit être réduite à l’essentiel : toute complexité visible le fait décrocher.

Adéquation produit. Le profil “balanced”, la mixité des modes (voiture partielle + TC) et l’affichage de l’économie CO2 vs voiture répondent à ses gains. Friction principale identifiée : l’interface doit rester aussi simple qu’un plan de métro. Toute option avancée visible sans qu’il l’ait demandée est un obstacle.

1.5 KPIs de succès

Les indicateurs suivants constituent le référentiel d’acceptation du MVP. Ils sont vérifiables par des outils ou des métriques objectives.

Performance technique

Indicateur	Cible	Méthode de mesure
Temps de réponse itinéraire (p95)	< 2 s	Logs Express (timestamp requête/réponse)
Temps de réponse itinéraire (p50)	< 800 ms	Idem
Score Lighthouse Performance	> 90	Audit Lighthouse DevTools
Bundle frontend compressé (gzip)	< 300 ko	Rapport Vite build + vite-bundle-visualizer

Accessibilité et qualité

Indicateur	Cible	Méthode de mesure
Erreurs bloquantes Axe DevTools	0	Audit Axe manuel sur les 5 écrans principaux
Ratio de contraste (texte normal)	>= 4.5:1	Axe + vérification palette UrbanFlow
Navigation clavier complète	Toutes les fonctionnalités atteignables	Test manuel

Couverture fonctionnelle

Indicateur	Cible
Fonctionnalités incluses au périmètre MVP	Authentification, planificateur multimodal, gamification et boutique de récompenses, tableau de bord citoyen
Consultation hors ligne (usager)	App shell, derniers itinéraires calculés et tuiles consultées disponibles sans réseau (service worker PWA + IndexedDB)

Impact métier (projection, non mesurable en MVP)

Ces indicateurs sont des projections destinées à la collectivité. Ils fondent le ROI non monétaire du projet.

- Taux de trajets reportés vers les modes doux (objectif Nantes Métropole : +15% d’ici 2030).
- Économie CO2 moyenne par usager actif par mois (calculée depuis les facteurs ADEME et l’historique de trajets).
- Nombre de trajets voiture évités (dérivé du report modal mesuré).

Ces trois métriques sont calculables dès que la solution est déployée en production et que la base d’utilisateurs est suffisante pour être statistiquement significative. Leur définition dès le MVP permet de préparer les tableaux de bord opérationnels V2 sans refactoring du modèle de données.

2. Pilotage, analyse et gestion du projet

2.1 Comparatif des solutions techniques

Nantes Métropole impose trois contraintes simultanées : une application installable sur terminal mobile sans passer par un store applicatif, un calcul d'itinéraire multimodal en temps réel, et la maîtrise souveraine des données de géolocalisation. Trois familles d'approches ont été étudiées et soumises à une grille multicritères fondée sur les douze contraintes officielles du projet.

Critère	Application native (Flutter / Kotlin)	Plateforme low-code (Bubble, Glide)	PWA React + TypeScript (retenu)
C1 - PWA installable	Incompatible - distribution via App Store / Play Store ; exclut les terminaux sans droits d'installation de store	Support partiel, limité aux fonctionnalités du builder ; aucune garantie d'installation hors store	Natif - manifest + service worker, installation depuis le navigateur sans intermédiaire
C2 - Responsive et UX	UX native excellente par plateforme, mais deux codebases (iOS + Android) ; charge doublée pour un résultat comparable sur mobile	Responsive limité aux templates du builder, personnalisation UX bloquée sur le scoring et les interactions métier	Tailwind mobile-first, design system UrbanFlow (mode clair et sombre), contrôle total de l'UX ; un seul codebase
C3 - Normes et standards	Solide (langages standard Flutter/Kotlin), mais hors standards web ouverts	Verrouillage éditeur (vendor lock-in), export limité ; normes web non respectées	Standards web ouverts : HTML5, ES2022, API Service Worker, TypeScript strict, ESLint
C4 - Sécurité OWASP	Maîtrise possible côté API, mais sécurité du store dépendante de l'éditeur de plateforme	Sécurité non auditable : dépendance totale à l'éditeur ; conformité OWASP non vérifiable	Helmet, rate-limit, CORS, Zod, bcrypt, JWT cookie HttpOnly ; audit OWASP complet maîtrisé
C5 - Éco-conception	Binaires volumineux (Android + iOS), deux versions à maintenir, build CI double	Plateforme tierce consommatrice de ressources ; aucun contrôle du bundle ni des assets	Contrôle total du bundle (cible < 300 ko gzip), lazy loading des routes, tree-shaking natif Vite
C6 - Géolocalisation	API GPS native pleine capacité, mais données de localisation transitent par les services tiers du store	Géoloc disponible via l'API du builder, sans contrôle du flux de données ni du consentement utilisateur	API navigateur standard, consentement explicite contrôlé dans l'app, aucune donnée GPS transmise à un tiers
C7 - Accessibilité WCAG 2.1 AA	Dépend du framework UI retenu ; audit non transposable aux outils web standard (Axe DevTools)	Non auditable via les outils web standard ; conformité WCAG non garantie par le builder	Navigateur standard : audit Axe DevTools complet, navigation clavier, ARIA, contrastes $\geq 4.5:1$
C8 - RGPD	Maîtrise possible si hébergement propre, mais distribution via store implique des tiers (analytics store)	Données hébergées chez l'éditeur tiers : incompatible avec la clause de souveraineté ; RGPD non maîtrisable	Hébergement choisi, zéro partage de données personnelles ; consentement géoloc, droit à l'effacement (<code>DELETE /api/users/me</code>)
C9 - Interopérabilité	Compatible avec les APIs REST, mais couplage fort si changement de provider de transport	Limité aux connecteurs natifs du builder ; ajout d'un provider non standard impossible	Fetch standard, pattern Stratégie (TransportProvider), OpenAPI 3.0 (Swagger) ; ajout de provider sans modifier le module routing
C10 - Performances mobilité	Performance native excellente par plateforme, mais binaires lourds et gestion mémoire spécifique à chaque OS	Performances dépendantes du runtime du builder, sans contrôle sur le parallélisme des appels réseau	Appels providers parallélisés (<code>Promise.all()</code>), cache Zustand côté client, service worker offline ; cible Lighthouse Performance > 90
C11 - Sécurité des données de déplacement	Stockage local sécurisé par l'OS, mais les données transitent par les services analytics et crash reporting du store	Sécurité dépendante de l'éditeur tiers ; chiffrement et gestion des accès aux données de trajet non auditable	HTTPS forcé, authentification JWT sur toutes les routes de données, coordonnées GPS arrondies à 4 décimales avant transmission aux providers
C12 - Normes accessibilité transport	Dépend du framework UI retenu ; intégration d'un filtre PMR nécessite un connecteur natif par plateforme	Filtre PMR non paramétrable finement, limité aux blocs disponibles dans le builder	Filtre PMR intégré au moteur de scoring : seuil de marche réduit à 5 minutes, pénalité renforcée sur les segments vélo

L'approche native est écartée pour deux raisons structurelles. La distribution par store est incompatible avec la contrainte d'installation sans intermédiaire. Le maintien de deux codebases dépasse la capacité d'un projet solo de 12 semaines pour un gain fonctionnel nul par rapport à une PWA (Progressive Web App, application web installable sur l'écran d'accueil depuis le navigateur, sans passer par un store applicatif).

L'approche low-code est écartée pour verrouillage éditeur et absence de contrôle des données personnelles : incompatible avec la contrainte C8 et la clause de souveraineté portée par Nantes Métropole. Elle rend également impossible l'implémentation du moteur

de scoring personnalisé.

La PWA React + TypeScript est retenue. Elle satisfait les douze contraintes, repose sur des standards web ouverts et garantit un contrôle complet du bundle, de la sécurité et des données utilisateur.

Moteur d'optimisation multicritères et non apprentissage automatique

La demande de Nantes Métropole mentionne une « IA qui optimise les trajets ». L'équipe retient un **moteur d'optimisation multicritères déterministe** plutôt qu'un modèle d'apprentissage automatique (machine learning). Ce choix repose sur trois arguments que l'équipe considère comme non négociables.

Éco-conception (C5). L'entraînement et l'inférence d'un modèle de machine learning génèrent une empreinte carbone significative. Proposer une solution d'aide à la mobilité durable en s'appuyant sur une technologie énergivore est en contradiction directe avec l'objet du produit.

Explicabilité et RGPD (C8). Un algorithme déterministe est intégralement auditable : chaque itinéraire recommandé peut être expliqué à l'utilisateur par ses pondérations et ses facteurs (durée, CO2, confort). Le RGPD (Règlement Général sur la Protection des Données) impose la transparence sur les décisions automatisées susceptibles d'affecter les personnes. Un modèle boîte noire ne satisfait pas cette exigence.

Disponibilité des données. Un modèle supervisé requiert un jeu de données d'entraînement calibré sur les comportements de déplacement nantais. Ce jeu n'existe pas en open data à l'échelle nécessaire pour un MVP. L'algorithme déterministe pondéré est reproductible et fiable dès la première démonstration.

Le besoin réel derrière la formulation « IA » est une recommandation pertinente selon plusieurs critères pondérés en temps réel. C'est précisément ce que le moteur multicritères délivre, sans coût écologique ni risque réglementaire.

Une couche de personnalisation par historique (adaptation des pondérations au fil des trajets de l'utilisateur) est envisageable en V2, explicitement hors périmètre MVP pour les raisons ci-dessus.

2.2 Justification des choix techniques

Chaque technologie retenue a été mise en regard d'au moins une alternative. Le tableau suivant documente ces arbitrages selon le triptyque besoin, options, choix retenu.

Choix retenu	Alternative écartée	Justification
React 18 + TypeScript strict	Vue 3, Angular 17	Typage statique strict réduit les erreurs à la compilation ; cohérence front/back via les types partagés dans shared/types/ ; écosystème éprouvé
Vite	Create React App, Webpack	Build 10 à 20 fois plus rapide, HMR (Hot Module Replacement) instantané, tree-shaking natif, bundle minimal (C5)
TailwindCSS v4	CSS-in-JS (Emotion), Sass	Zéro runtime CSS, purge automatique des classes inutilisées, bundle de styles inférieur à 1 ko (C5)
Zustand	Redux, Context API	Légereté (moins de 1 ko gzip), absence de boilerplate ; Redux introduit un volume de code disproportionné pour le périmètre d'un MVP solo
Leaflet.js + tuiles CartoDB Positron	Google Maps Platform	Leaflet pèse environ 40 ko contre plus de 200 ko pour le SDK Google Maps ; aucune carte bancaire requise ni quota d'usage ; cartographie open source (C5)
Recharts	Chart.js, D3.js	Composants React déclaratifs, tree-shakable, sans manipulation DOM impérative
Node.js + Express + TypeScript	NestJS, Fastify	Principe KISS (Keep It Simple, Stupid) : Express est suffisant pour un monolithe modulaire ; NestJS ajoute une couche de framework non justifiée par le périmètre
PostgreSQL + PostGIS	MongoDB, Redis seul	Requêtes géospatiales natives (calcul de distance, arrêts dans un rayon en mètres), modèle relationnel pour l'intégrité des données utilisateur et historique de trajets
Zod (validation des entrées)	Joi, express-validator	Inférence des types TypeScript depuis le schéma de validation : source de vérité unique, partagée entre le frontend et le backend via shared/
JWT court (15 min) + refresh token cookie HttpOnly	Session serveur, token en localStorage	Stateless ; cookie HttpOnly protège contre les attaques XSS (Cross-Site Scripting) ;

Choix retenu	Alternative écartée	Justification
		SameSite=Strict protège contre le CSRF (C4, C11)
Pattern Stratégie (TransportProvider)	Appels directs aux APIs dans routing.service	Évolutivité : ajouter un mode de transport ou un opérateur ne modifie pas le module de routing, il suffit d'implémenter l'interface TransportProvider (C9)
Monolithe modulaire	Microservices	Principe YAGNI (You Aren't Gonna Need It) : les microservices apportent un surcoût opérationnel injustifié pour un MVP solo. L'architecture modulaire permet d'extraire un service indépendant si le besoin de scalabilité apparaît en V2
APIs publiques (Transitous, OSRM)	Auto-hébergement OTP ou OSRM	L'auto-hébergement d'un moteur de routage complet représente plusieurs jours d'infrastructure pour un gain marginal en prototype ; les APIs publiques suffisent à la démonstration
Vercel + Render + Supabase	VPS auto-géré (OVH, Scaleway)	CI/CD intégrée, HTTPS automatique, free tiers suffisants pour le prototype ; architecture portable via Docker si migration nécessaire

2.3 Méthodologie

Le projet est conduit en Scrum adapté au contexte solo. La structure Scrum classique est conservée (sprints délimités, backlog priorisé, revue à chaque fin de sprint), et les rituels non adaptables à un contexte solo sont remplacés par des équivalents fonctionnels.

Adaptation des rituels Scrum

Rituel Scrum classique	Adaptation solo
Daily stand-up	Journal de bord quotidien (fichier texte versionné) : blocages identifiés, décisions prises, objectif du lendemain
Sprint review	Démonstration autonome : parcours complet de la fonctionnalité dans le navigateur, capture ou note de validation formelle
Rétrospective	30 minutes en fin de sprint : une friction identifiée, une amélioration actionnée dans le sprint suivant
Planning poker	Estimation par points de complexité (1, 3, 5, 8), ajustée à la charge estimée en jours

Le backlog est géré dans un tableau Kanban (colonnes : Backlog, En cours, Revu, Terminé) accessible à l'équipe. La priorisation suit la méthode MoSCoW (Must have, Should have, Could have, Won't have) : les fonctionnalités classées "Must" constituent le périmètre MVP ; les "Won't" constituent la V2 documentée.

Rôles portés en solo

Un projet de cette envergure mobilise des compétences habituellement réparties sur une équipe. Le tableau suivant nomme explicitement les casquettes et la répartition de charge estimée. En production réelle, ces rôles seraient distribués sur une équipe de 4 à 6 personnes.

Rôle	Responsabilités principales	Charge estimée
Product Owner	Priorisation du backlog, critères d'acceptation, relation client	10 %
Lead Développeur frontend	Composants React, design system, accessibilité	30 %
Lead Développeur backend	API Express, routage, scoring, authentification	30 %
UX/UI	Wireframes, prototype Figma, charte graphique	10 %
DevOps	CI/CD GitHub Actions, déploiement Vercel / Render / Supabase	5 %
QA / Référent accessibilité	Tests Vitest, audit Axe DevTools, recette manuelle	10 %
Référent sécurité / RGPD	Relecture OWASP, conformité CNIL, consentement géolocalisation	5 %

2.4 Amélioration continue

La démarche DMAIC (Define, Measure, Analyze, Improve, Control) structure l'amélioration continue du projet. Elle est illustrée ici sur un cas anticipé dès l'analyse architecturale : la gestion de la latence de calcul d'itinéraire, qui conditionne directement le KPI p95 < 2 s fixé en section Contexte.

Define. Le planificateur multimodal doit répondre en moins de 2 secondes à p95 sur mobile. L'analyse de l'architecture de routing identifie un risque de dépassement : dans un schéma naïf, les appels aux deux providers de transport (TransitousProvider pour les transports en commun, OsmrProvider pour les modes actifs) sont séquentiels. Le temps total est alors la somme de leurs latences individuelles.

Measure. L'équipe prévoit de mesurer les temps de réponse sur 50 requêtes représentatives couvrant des trajets nantais variés (Doulon, Vertou, Commerce, Île de Nantes), avec horodatage côté serveur en entrée et sortie du module de routing. Les indicateurs suivis sont p50, p95 et p99. Avant optimisation, l'estimation basée sur les latences moyennes connues des APIs (Transitous : 800 ms à 1,5 s selon la complexité de la requête ; OpenWeatherMap : 300 à 600 ms) projette un p95 supérieur à 3 secondes sur des appels séquentiels.

Analyze. La cause racine est identifiée depuis la conception : les appels aux providers sont séquentiels alors qu'ils sont fonctionnellement indépendants. Aucun résultat d'un provider n'est nécessaire pour déclencher l'appel suivant. La dépendance séquentielle est accidentelle, pas fonctionnelle.

Improve. Deux mesures sont retenues. La parallélisation des appels via `Promise.all()` : TransitousProvider et OsmrProvider sont sollicités simultanément depuis routing.service, le résultat agrégé attend la complétion du plus lent. Un cache mémoire (Map JavaScript, TTL de 10 minutes) sur les données météo OpenWeatherMap élimine les appels redondants pour des requêtes portant sur le même secteur géographique dans la même fenêtre de temps. Cette combinaison cible un p50 inférieur à 800 ms et un p95 inférieur à 2 s.

Control. Chaque requête de calcul d'itinéraire sera horodatée côté serveur. Un seuil d'alerte est fixé à p95 > 2 s : tout dépassement constaté en recette manuelle déclenche une investigation avant le merge suivant dans la branche principale. La mesure de performance reste hors du pipeline d'intégration continue, volontairement limité au lint et au build pour rester stable sur des runners partagés, comme justifié dans la section consacrée à la qualité et aux tests.

Kaizen continu

Au-delà du cas DMAIC, chaque sprint intègre une revue d'amélioration de 30 minutes. L'objectif est d'identifier une friction de process ou de code et d'y apporter une réponse dans le sprint suivant. Les principes DRY (Don't Repeat Yourself, ne pas dupliquer la logique) et SRP (Single Responsibility Principle, un module fait une seule chose) font l'objet d'une relecture systématique avant chaque merge. À titre d'illustration : l'extraction d'un hook `useJourneySearch` pour éliminer la duplication de logique de requête entre composants, l'ajout d'un template GitHub Issue pour standardiser les rapports de bug, ou encore l'introduction de la validation Zod en middleware plutôt qu'en controller.

2.5 Planning et charge

Le chiffrage détaillé en section Budget évalue la charge de réalisation à 83 jours, décomposés par rôle spécialisé (cadrage, développement frontend, développement backend, tests/accessibilité/sécurité, DevOps, gestion de projet) et valorisés au tarif d'une équipe professionnelle de 4 à 6 personnes. Porté en solo, ce chiffrage se compresse : l'absence de réunions de coordination inter-rôles, de documents de passation entre équipiers et de changement de contexte entre spécialités réduit la charge réellement consommée. Le planning solo cible 60 jours ouvrés sur 12 semaines, soit une compression d'environ 28 % par rapport au chiffrage équipe, cohérente avec le gain de productivité d'un développeur unique qui maîtrise l'ensemble de la pile applicative sans dépendance de synchronisation.

Le projet est découpé en six sprints de deux semaines (le planificateur multimodal, périmètre le plus dense, en occupe quatre), soit douze semaines de développement au total. Le rendu du dossier intervient le 20/07, à la clôture du sprint planificateur, tandis que les sprints suivants (gamification, contraintes transversales, stabilisation) poursuivent l'exécution du projet jusqu'à la démonstration de soutenance.

Sprint	Période	Périmètre fonctionnel	Jalon
S1	08/06 - 21/06	Mise en place de l'infrastructure, authentification (inscription, connexion, JWT, refresh token), modèle de données initial	Authentification fonctionnelle en local (21/06)
S2-S3	22/06 - 19/07	Planificateur multimodal complet : TransitousProvider (bus, tramway, navibus, train), OsmrProvider (vélo, marche, scooter), moteur de scoring multicritères, filtres PMR, météo OpenWeatherMap	Scoring end-to-end fonctionnel (19/07)
S4	20/07 - 02/08	Gamification : attribution de points par trajet, badges, tableau de bord citoyen, historique de trajets, boutique de récompenses	Parcours gamification et récompenses complet (02/08)

Sprint	Période	Périmètre fonctionnel	Jalon
S5	03/08 - 16/08	Contraintes transversales : accessibilité WCAG 2.1 AA, éco- conception (bundle, lazy loading), sécurité OWASP, RGPD (consentement géolocalisation, suppression du compte)	Audit Axe DevTools : 0 erreur bloquante (16/08)
S6	17/08 - 30/08	Stabilisation, mode démo (bascule DEMO_MODE), couverture de tests Vitest, documentation Swagger / OpenAPI, recette finale	Prototype complet, prêt pour la démonstration (30/08)

Le rendu du dossier, le 20/07, précède donc les preuves qu’il annonce pour les contraintes transversales (audit Axe DevTools, rapport Lighthouse, couverture Vitest, rapport de bundle) : ces captures sont insérées dans une révision du dossier une fois les sprints S5 et S6 clôturés, avant la soutenance de septembre.

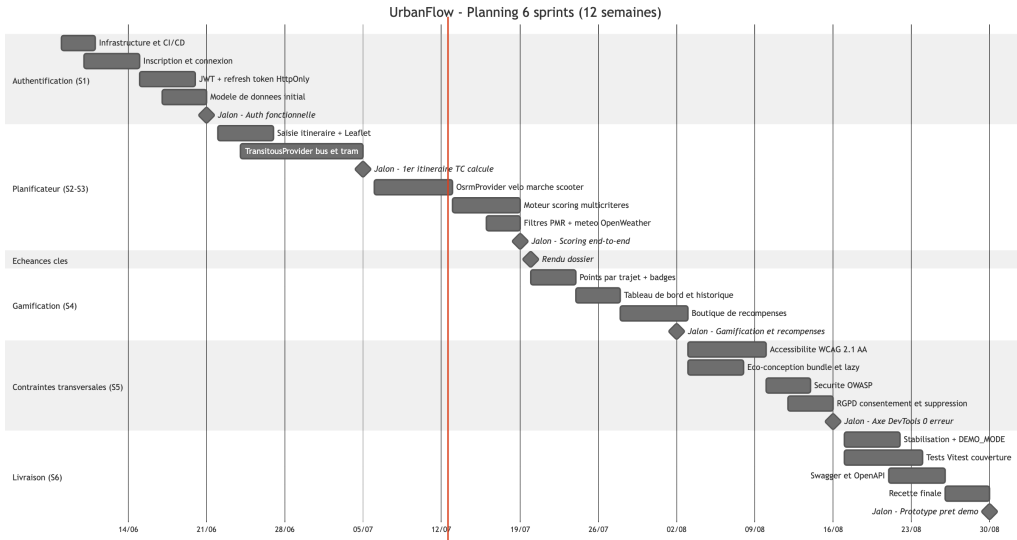


Diagramme de Gantt

2.6 Budget et chiffrage

Coût de réalisation

Le chiffrage est établi selon la méthode charge estimée multipliée par le TJM (taux journalier moyen) de marché pour un profil développeur full-stack senior en région Pays-de-la-Loire (source : grille salariale APEC 2024). Il permet à Nantes Métropole d'évaluer le coût de reconduction ou d'extension du projet avec une équipe professionnelle.

Poste	Charge estimée	TJM indicatif	Coût estimé
Cadrage, conception, UX/UI	10 j	450 EUR	4 500 EUR
Développement frontend (React, composants, design system)	25 j	500 EUR	12 500 EUR
Développement backend (API, routing, scoring, authentification)	25 j	500 EUR	12 500 EUR
Tests, accessibilité, sécurité	10 j	450 EUR	4 500 EUR
DevOps et déploiement (CI/CD, environnements)	5 j	500 EUR	2 500 EUR
Gestion de projet et documentation	8 j	450 EUR	3 600 EUR
Total réalisation	83 j		40 100 EUR

Ces TJM sont indicatifs. Le projet est conduit dans un cadre pédagogique par un seul développeur ; ce tableau matérialise la valeur marchande du travail proposé et constitue la base d'un chiffrage réaliste pour une reconduction en contexte professionnel.

Coût de fonctionnement annuel

Le TCO (Total Cost of Ownership, coût total de possession sur un an) couvre les charges récurrentes nécessaires à l'exploitation de la solution en production réelle, au-delà du prototype.

Poste	Coût annuel estimé
Hébergement (Vercel, Render, Supabase, tiers payants en production)	600 à 1 500 EUR
APIs externes (OpenWeatherMap, quota production)	200 à 600 EUR selon volume d'usage
Nom de domaine et certificats TLS	50 EUR
Maintenance corrective et évolutive (2 jours par mois minimum)	12 000 EUR
TCO annuel indicatif	12 850 à 14 150 EUR

Modèle économique

UrbanFlow est un service public numérique : gratuit pour le citoyen, financé par la collectivité. Trois leviers de financement sont identifiés.

Subventions fléchées transition écologique. L'ADEME (Agence de la transition écologique) et le programme Fonds Vert de l'État financent les projets de mobilité durable des collectivités. UrbanFlow répond aux critères d'éligibilité (réduction de l'empreinte carbone des déplacements, intermodalité, données ouvertes).

Mutualisation inter-collectivités. Le code source produit est réutilisable pour d'autres métropoles françaises disposant d'un réseau de transport comparable et d'un flux GTFS ouvert. Une mise en commun des coûts de maintenance réduit le TCO par collectivité.

ROI non monétaire. Le retour sur investissement d'un service public de mobilité ne se mesure pas en chiffre d'affaires. Il se mesure en externalités positives quantifiables : tonnes de CO2 évitées par report modal, réduction de la congestion aux points de comptage, données d'aide à la décision pour les politiques de mobilité. Ces indicateurs, définis en section A5, sont calculables dès le déploiement en production et constituent le rapport d'impact que Nantes Métropole peut présenter aux institutions financeuses et à ses administrés.

3. Conception : diagrammes UML

Trois diagrammes UML (Unified Modeling Language, langage de modélisation unifié) structurent la représentation de la solution. Le diagramme de cas d'utilisation délimite le périmètre fonctionnel et identifie les acteurs. Le diagramme de séquences détaille le flux de calcul d'itinéraire, du geste utilisateur à la réponse triée. Le diagramme de communication représente les mêmes échanges depuis une perspective structurelle, en mettant en avant les collaborations entre objets plutôt que leur chronologie.

3.1 Cas d'utilisation

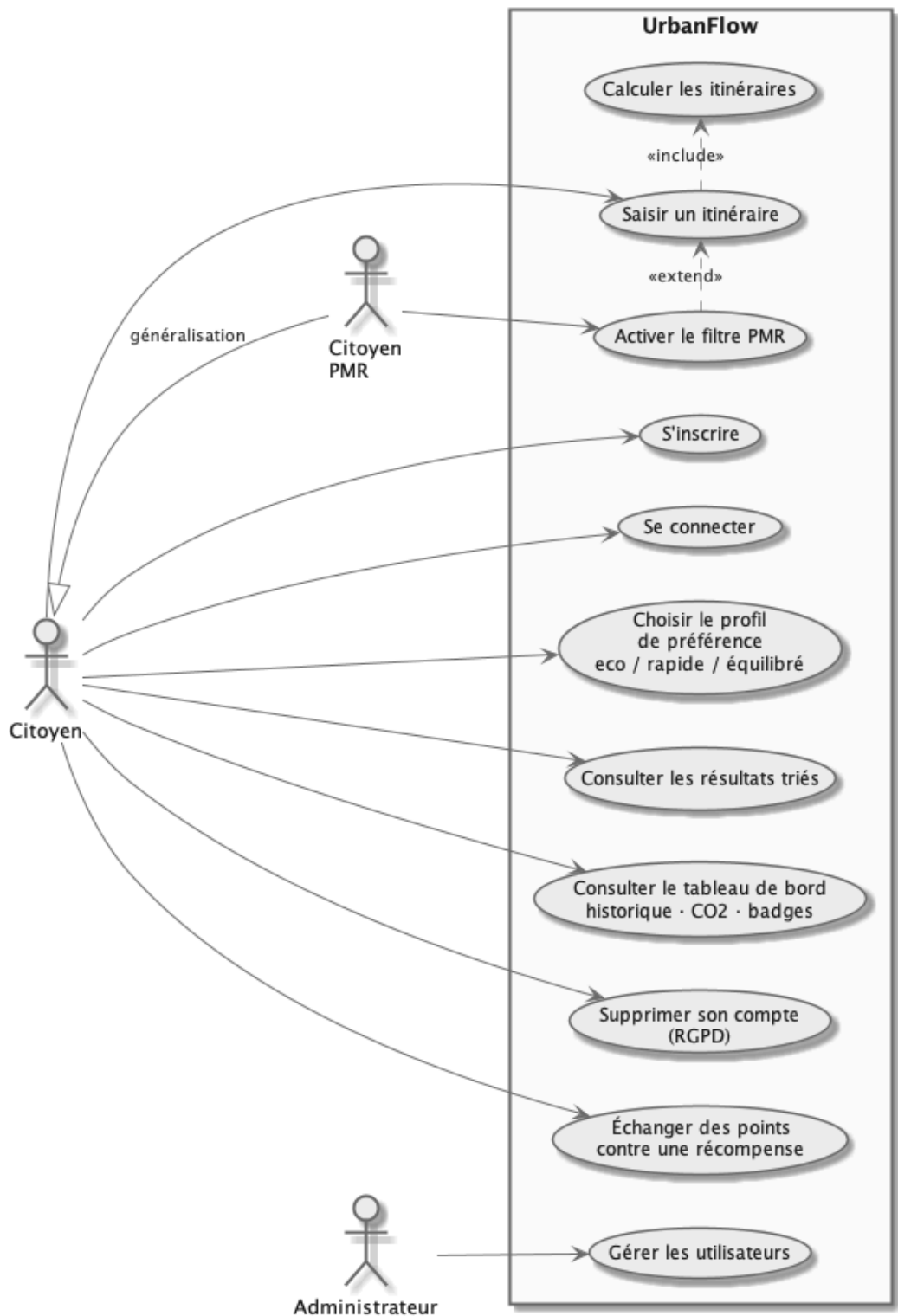


Diagramme de cas d'utilisation

Le diagramme implique trois acteurs et délimite le périmètre fonctionnel retenu pour le MVP.

Le Citoyen est l'acteur principal. Il s'inscrit, se connecte, saisit un itinéraire, sélectionne un profil de préférence parmi trois options (eco, rapide, équilibré) et consulte les résultats triés par score multicritères. Il accède à son tableau de bord, qui regroupe trois cas distincts : consultation de l'historique de trajets, consultation du score CO2 cumulé, et consultation des badges et points acquis via la gamification. Il peut également échanger les points accumulés contre une récompense du catalogue proposé par des partenaires

locaux (Bicloo, Naolib, sites culturels nantais) : ce cas débite son solde de points et génère un code d'échange unique. Le cas "Supprimer son compte" est directement accessible à tout citoyen authentifié, en application du droit à l'effacement prévu par le RGPD.

Le Citoyen-PMR (personnes à mobilité réduite) est une spécialisation du Citoyen par relation de généralisation. Il hérite de l'ensemble de ses cas d'utilisation et en ajoute un seul : "Activer le filtre PMR". Ce filtre modifie le comportement du moteur de scoring, en réduisant le seuil de marche toléré à cinq minutes et en appliquant une pénalité sur les segments vélo.

L'Administrateur est un acteur interne. Son seul cas dans le périmètre MVP est la gestion des comptes utilisateurs ; il n'accède à aucune donnée de déplacement individuelle.

Deux relations structurent le diagramme. La relation include lie "Saisir un itinéraire" à "Calculer les itinéraires" : le calcul est systématiquement déclenché à la soumission du formulaire. La relation extend lie "Activer le filtre PMR" à "Saisir un itinéraire" : c'est une extension optionnelle, conditionnée à l'activation explicite par l'utilisateur dans son profil. Cette distinction materialise la règle de gestion fondamentale du planificateur : le système calcule toujours un itinéraire, mais adapte ses filtres et pondérations selon le profil actif.

3.2 Séquences

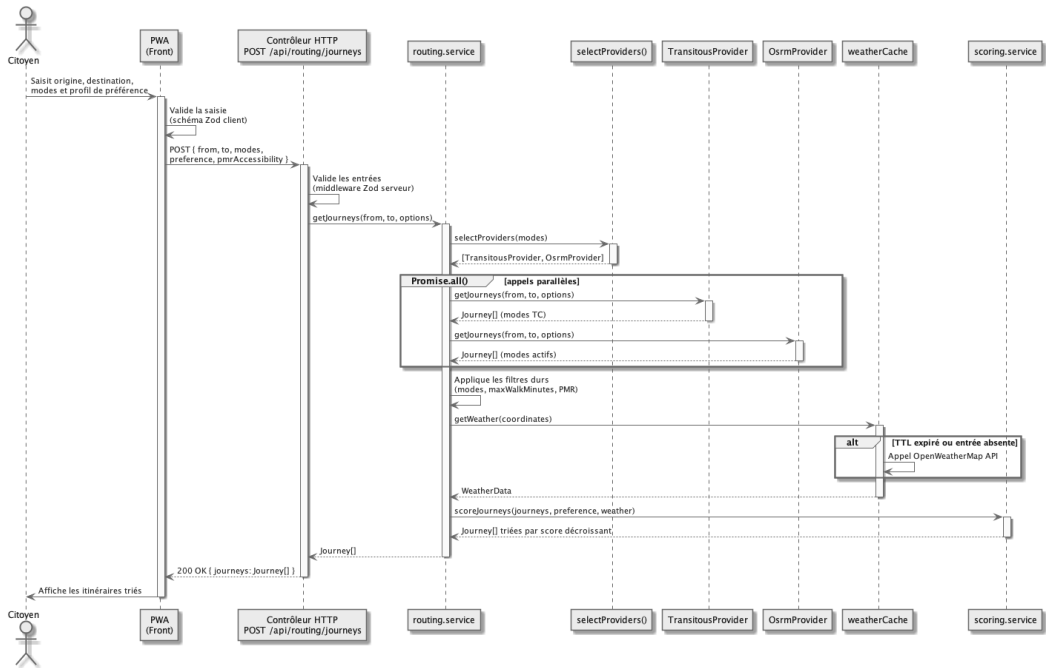


Diagramme de séquences

Le diagramme de séquences détaille le flux complet d'un calcul d'itinéraire, depuis la saisie utilisateur jusqu'à l'affichage des résultats triés. Il implique neuf participants : l'acteur citoyen, le frontend PWA, le contrôleur HTTP du module de routing, le service de routing, le sélecteur de providers, les deux providers de transport, le cache météo et le service de scoring.

La double validation des entrées apparaît en ouverture du flux. Le frontend valide la saisie via un schéma Zod côté client avant l'envoi, et le middleware serveur revalide les mêmes données à réception. Cette redondance est intentionnelle : elle applique le principe de défense en profondeur recommandé par l'OWASP (Open Web Application Security Project, référentiel de sécurité des applications web) et protège le backend même si le client est contourné.

La délégation à `selectProviders()` illustre le pattern Stratégie documenté dans l'architecture. `routing.service` ne connaît jamais directement les APIs externes. Il soumet les modes demandés au sélecteur, qui retourne la liste des implémentations de `TransportProvider` pertinentes : `TransitousProvider` pour les modes de transport en commun (bus, tramway, navibus, train), `OsmrProvider` pour les modes actifs (vélo, marche, scooter électrique).

Le bloc parallèle `Promise.all()` matérialise l'optimisation de latence identifiée par la démarche DMAIC présentée dans la section Pilotage. Les deux providers sont interrogés simultanément. La latence totale est celle du plus lent des deux, et non leur somme.

Le cache météo apparaît comme un participant distinct, doté d'une logique de décision propre : si la donnée météo pour les coordonnées demandées est présente et récente (TTL de dix minutes), l'appel à OpenWeatherMap est élimé. Cette logique de cache réduit les dépendances réseau aux API externes et maintient le KPI de temps de réponse p95 inférieur à deux secondes.

Enfin, `scoring.service` reçoit les itinéraires filtrés, les données météo et le profil de préférence, applique la formule multicritères et retourne les itinéraires triés par score décroissant. Le frontend reçoit une liste ordonnée et l'affiche sans tri supplémentaire, conformément au principe de séparation des responsabilités entre couches.

3.3 Communication

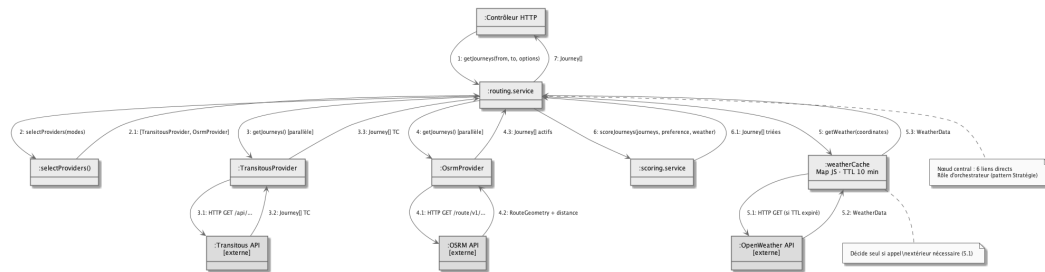


Diagramme de communication

Le diagramme de communication représente les mêmes échanges que le diagramme de séquences, depuis une perspective structurelle. L'axe du temps disparaît. Les participants deviennent des nœuds reliés par des liens de collaboration. L'ordre des messages est exprimé par une numérotation hiérarchique apposée sur chaque flèche.

La lecture de ce diagramme révèle trois propriétés architecturales que la séquence ne met pas aussi nettement en évidence.

Premièrement, `routing.service` est le nœud central de la collaboration. Il reçoit la requête du contrôleur (message 1), délègue la sélection des providers (2), déclenche les appels parallèles (3 et 4), interroge le cache météo (5) et délègue le scoring (6). Cette concentration de liaisons illustre le rôle d'orchestrateur que l'architecture lui attribue. Aucune autre couche du système ne détient autant de connexions directes.

Deuxièmement, `TransitousProvider` et `OsmProvider` ne communiquent jamais entre eux. Chacun entretient deux relations : une vers `routing.service` (qui le sollicite et reçoit sa réponse) et une vers son API externe. Cette indépendance est l'objectif premier du pattern Stratégie : ajouter un troisième provider, un opérateur de covoiturage par exemple, ne modifie ni `TransitousProvider` ni `OsmProvider`, ni `routing.service`.

Troisièmement, `weatherCache` apparaît comme un objet à part entière, distinct de l'API OpenWeatherMap. Il reçoit les requêtes de `routing.service` (5) et décide seul si un appel externe est nécessaire (5.1). Cette séparation matérialise la règle d'éco-conception posée dans les contraintes du projet : ne jamais appeler une API externe si la donnée est déjà disponible en mémoire.

4. Spécifications du planificateur multimodal

Le planificateur multimodal est la fonctionnalité centrale de la solution. Il agrège les itinéraires issus de plusieurs providers de transport, applique des filtres selon le profil utilisateur et trie les résultats via un moteur de scoring multicritères. Cette section détaille ses spécifications fonctionnelles et techniques.

4.1 Fonctionnelles

Parcours utilisateur nominal

L'utilisateur authentifié arrive sur la carte, centrée par défaut sur la place du Commerce (latitude 47.218, longitude -1.553, centre de Nantes Métropole). Un bouton de localisation lui propose d'utiliser sa position GPS. S'il accepte, le champ "départ" se remplit automatiquement via l'API de géolocalisation du navigateur. Il peut aussi saisir une adresse manuellement.

L'utilisateur saisit sa destination, sélectionne les modes souhaités parmi sept options (bus, tramway, navibus, train, vélo, marche, scooter électrique), choisit un profil de préférence (eco, rapide, équilibré) et soumet le formulaire. La validation côté client s'exécute avant tout envoi réseau. Si un champ obligatoire est absent ou mal formé, un message d'erreur précis apparaît directement sous le champ concerné.

Le système calcule les itinéraires et renvoie une liste triée par score décroissant, affichée dans un panneau glissant au-dessus de la carte. Chaque itinéraire présente la durée totale, la distance, le score CO2, l'économie en grammes comparée à un trajet en voiture, et les modes représentés par des icônes. L'utilisateur sélectionne un itinéraire pour en consulter le détail : liste des segments, horaires de correspondance et tracé sur la carte.

User stories et critères d'acceptation

Les critères d'acceptation suivent le format Gherkin (langage structuré Étant donné / Quand / Alors, permettant d'exprimer des scénarios de test lisibles par toutes les parties prenantes).

US-01. Calcul d'itinéraire multimodal

En tant que citoyen authentifié, je veux saisir une origine et une destination pour obtenir une liste d'itinéraires multimodaux triés.

```
Scénario : calcul réussi
  Étant donné que je suis connecté
  Et que j'ai saisi une adresse de départ et une adresse d'arrivée valides
  Et que j'ai sélectionné au moins un mode de transport
  Quand je sou mets le formulaire de planification
  Alors le système affiche au moins un itinéraire dans les 2 secondes
  Et les itinéraires sont triés par score décroissant selon le profil actif
```

```
Scénario : champ obligatoire manquant
  Étant donné que le champ "destination" est vide
  Quand je tente de soumettre le formulaire
  Alors un message "Destination requise" s'affiche sous le champ concerné
  Et aucun appel réseau n'est déclenché
```

US-02. Profil de préférence

En tant que citoyen, je veux choisir un profil de préférence pour que les itinéraires proposés correspondent à ma priorité du moment.

```
Scénario : profil eco
  Étant donné que j'ai sélectionné le profil "eco"
  Quand les résultats sont affichés
  Alors les itinéraires sont triés par score pondéré où le critère CO2 pèse 70 % du score final
  Et l'itinéraire en tête de liste porte le badge de profil "Eco"
```

Le profil "eco" ne garantit pas que l'itinéraire le plus sobre en CO2 apparaisse toujours en tête : la pondération conserve 20 % de poids sur la durée et 10 % sur le confort, de sorte qu'un itinéraire légèrement moins sobre mais nettement plus rapide peut le devancer. C'est le comportement attendu d'un moteur multicritères, qui arbitre entre plusieurs objectifs plutôt que d'optimiser un seul critère de façon absolue.

US-03. Filtre accessibilité PMR

En tant que citoyen à mobilité réduite, je veux activer un filtre PMR pour n'obtenir que des itinéraires compatibles avec mes contraintes de déplacement.

```
Scénario : filtre PMR actif, itinéraires compatibles disponibles
  Étant donné que j'ai activé le filtre PMR dans mon profil
  Quand je calcule un itinéraire
  Alors aucun itinéraire retourné ne comporte un segment de marche supérieur à 5 minutes
  Et les itinéraires comportant un segment vélo sont absents ou fortement pénalisés

Scénario : aucun itinéraire compatible PMR
  Étant donné que le filtre PMR est actif
```

Et qu'aucun itinéraire ne satisfait la contrainte de 5 minutes de marche maximum
Quand le calcul est terminé
Alors le système affiche le message "Aucun itinéraire accessible trouvé pour ce trajet"
Et propose à l'utilisateur d'assouplir ses filtres

US-04. Géolocalisation et saisie manuelle

En tant que citoyen, je veux utiliser ma position GPS pour préremplir le point de départ, et pouvoir saisir manuellement si je refuse la géolocalisation.

Scénario : permission de géolocalisation accordée
Étant donné que j'ai accordé la permission de géolocalisation
Quand j'ouvre le planificateur
Alors le champ "départ" est prérempli avec mes coordonnées GPS
Et une adresse approximative est affichée (géocodage inverse)

Scénario : permission de géolocalisation refusée
Étant donné que j'ai refusé la permission de géolocalisation
Quand j'ouvre le planificateur
Alors le champ "départ" reste vide et éditable
Et le message "Saisissez votre adresse de départ" invite à la saisie manuelle

US-05. Affichage de l'empreinte carbone

En tant que citoyen, je veux voir l'empreinte carbone de chaque itinéraire et l'économie réalisée par rapport à un trajet en voiture.

Scénario : affichage CO2
Étant donné que des itinéraires sont affichés
Quand je consulte la liste des résultats
Alors chaque itinéraire affiche son empreinte en grammes d'équivalent CO2
Et l'économie en grammes comparée au trajet équivalent en voiture particulière

Règles de gestion

Pondérations par profil de préférence

Le moteur applique trois coefficients dont la somme est toujours égale à 1.0.

Profil	Coefficient durée	Coefficient CO2	Coefficient confort
eco	0.2	0.7	0.1
rapide	0.7	0.2	0.1
équilibré	0.4	0.5	0.1

Filtres durs appliqués avant le scoring

Deux filtres éliminent des itinéraires avant que le calcul de score ne s'exécute. Le filtre de modes supprime tout itinéraire dont un segment utilise un mode non sélectionné par l'utilisateur. La marche est toujours tolérée, quel que soit le filtre actif, car elle est inévitable dans les correspondances. Le filtre `maxWalkMinutes` élimine les itinéraires dont un segment de marche dépasse le seuil configuré (valeur par défaut : 15 minutes). Quand le filtre PMR est actif, ce seuil est réduit à la valeur minimale entre `maxWalkMinutes` et 5 minutes.

Règle Bicloo

OsmrProvider refuse de construire un itinéraire incluant une station Bicloo si la station de départ la plus proche dépasse 1,5 km de marche depuis le point de départ. Ce seuil évite de proposer des segments de marche absurdes qui dégraderaient l'expérience utilisateur sans alternative viable.

Pénalités PMR dans le score de confort

Quand le filtre PMR est actif, la pénalité sur un segment de marche dépassant le seuil est portée à 60 points (contre 40 en mode standard). Tout itinéraire comportant un segment vélo reçoit une pénalité supplémentaire de 50 points sur le score de confort.

4.2 Techniques

Contrat d'API

Le planificateur expose un seul endpoint.

```
POST /api/routing/journeys
```

Authentification requise via un token JWT (JSON Web Token) dans le header `Authorization: Bearer <token>`.

Corps de la requête (validé par middleware Zod)

```
{
  from: {
```

```

lat: number, // entre 47.0 et 47.5 (emprise Nantes Métropole)
lng: number // entre -2.0 et -1.0
},
to: {
  lat: number,
  lng: number
},
modes: Array<'bus' | 'tramway' | 'navibus' | 'train' | 'bike' | 'walk' | 'scooter'>,
preference: 'eco' | 'fast' | 'balanced',
pmrAccessibility: boolean,
maxWalkMinutes?: number, // optionnel, défaut 15
departureTime?: string // optionnel, ISO 8601
}

```

Corps de la réponse (200)

```

{
  journeys: Array<{
    id: string,
    totalDurationMin: number,
    totalDistanceKm: number,
    totalCo2g: number,
    co2SavedVsCarG: number,
    score: number, // score final entre 0 et 100
    segments: Array<{
      mode: string,
      from: { lat: number, lng: number },
      to: { lat: number, lng: number },
      durationMin: number,
      distanceKm: number,
      co2g: number,
      geometry: GeoJSON.LineString, // GeoJSON : format ouvert d'échange de données géographiques
      line?: string, // ex. "C1" pour le tramway ligne 1
      direction?: string
    }>
  }>
}

```

Codes d'erreur

Code HTTP	Cas
400	Corps invalide : champ manquant, type incorrect, coordonnées hors zone
401	Token absent ou expiré
404	Aucun itinéraire trouvé pour les paramètres donnés
503	Les deux providers externes sont en timeout et DEMO_MODE est désactivé

Note sur OsmProvider

Le serveur public OSRM (Open Source Routing Machine, moteur de routage open source) ne propose que le profil de calcul routier voiture. OsmProvider récupère la géométrie et la distance depuis ce profil, puis calcule la durée de chaque segment depuis des vitesses constantes par mode : 15 km/h pour le vélo, 5 km/h pour la marche, 20 km/h pour le scooter électrique. Cette approximation est documentée et acceptable pour un MVP.

Algorithme de scoring multicritères

Le moteur est entièrement déterministe. Aucun composant statistique ni aucun modèle d'apprentissage n'entre dans son calcul.

Formule générale

$$\text{score_final} = (\text{w_durée} \times \text{score_durée}) + (\text{w_CO2} \times \text{score_CO2}) + (\text{w_confort} \times \text{score_confort})$$

Score de durée

La durée de référence est fixée à 120 minutes. Un itinéraire de 0 minute obtiendrait un score de 100 ; un itinéraire de 120 minutes ou plus obtient 0.

$$\text{score_durée} = \max(0, 100 - (\text{totalDurationMin} / 120) \times 100)$$

Score CO2

Le score CO2 compare l'empreinte de l'itinéraire à celle du trajet équivalent en voiture particulière. Le facteur voiture retenu est 253 g CO2e/km, issu de la Base Empreinte de l'ADEME (Agence de la transition écologique).

score_CO2 = max(0, (1 - totalCo2g / (totalDistKm × 253)) × 100)

Facteurs CO2 par mode (source : Base Empreinte ADEME, g CO2e/km par passager, phase d'usage) :

Mode	Facteur (g CO2e/km)
Voiture particulière (référence)	253
Bus	109
Tramway	4
Navibus	50
Train TER électrifié	14
Vélo	0
Marche	0
Scooter électrique	0

Ces facteurs couvrent uniquement la phase d'usage du déplacement, c'est-à-dire les émissions directes produites pendant le trajet, à l'exclusion de la fabrication du véhicule et de sa fin de vie (analyse de cycle de vie). C'est ce périmètre qui justifie un facteur nul pour le vélo, la marche et le scooter électrique : aucun de ces trois modes n'émet directement de CO2 pendant le déplacement. Une analyse de cycle de vie complète, hors périmètre MVP, attribuerait un facteur non nul à la fabrication d'un scooter électrique et de sa batterie.

L'empreinte totale d'un itinéraire est la somme des produits (distance du segment en km × facteur du mode) pour chaque segment. L'économie CO2 est la différence entre l'empreinte du trajet équivalent en voiture et l'empreinte calculée.

Score de confort

Le score de confort mesure le ratio de segments dont le mode est inclus dans les préférences de l'utilisateur, rapporté à 100. Des pénalités et un bonus ponctuel l'ajustent ensuite, selon les segments de l'itinéraire et, quand elle est disponible, la donnée météo du point de départ, conformément au diagramme de séquences qui transmet cette donnée au moteur de scoring. La météo provient du cache OpenWeatherMap décrit dans la section consacrée à l'architecture (TTL de 10 minutes).

Situation	Ajustement
Segment marche dépassant le seuil standard (15 min)	-40 points
Segment marche dépassant le seuil PMR (5 min)	-60 points
Segment vélo si filtre PMR actif	-50 points
Conditions humides (pluie, neige, orage) ou vent supérieur à 40 km/h, sur un itinéraire comportant un segment vélo	-30 points
Conditions humides (pluie, neige, orage), itinéraire entièrement en transport en commun (sans segment vélo)	+10 points

Le bonus par temps humide valorise les modes couverts (bus, tramway, navibus, train) quand la pluie dégrade le confort perçu des modes actifs. Il ne s'applique pas si l'itinéraire comporte un segment vélo, même combiné à un segment de transport en commun. Le score de confort reste borné entre 0 et 100 après application de l'ensemble des ajustements.

Modèle de données

Le planificateur interagit avec trois tables PostgreSQL. L'extension PostGIS est requise pour les requêtes géospatiales sur les coordonnées GPS. Le schéma complet, incluant les tables de gamification et de récompenses, est détaillé dans la section consacrée à l'architecture technique.

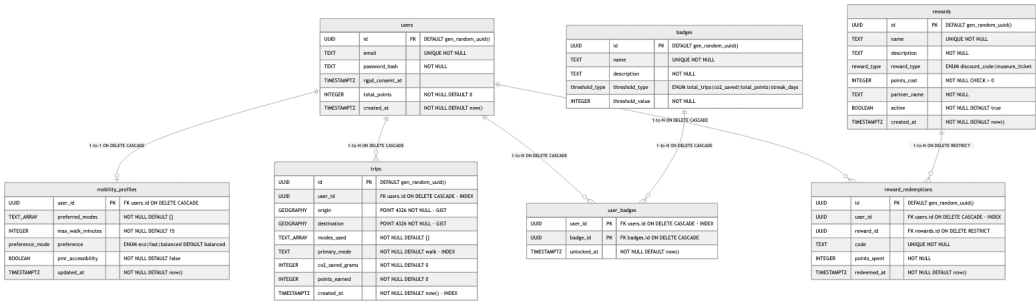


Schéma relationnel complet

Rendu du schéma entités-relations complet (ces trois tables et celles de la gamification), généré depuis le source Mermaid de la section consacrée à l'architecture technique.

Table `users` (partagée avec le module d'authentification)

Colonne	Type	Contrainte
id	UUID	PK, DEFAULT gen_random_uuid()
email	TEXT	UNIQUE, NOT NULL

Colonne	Type	Contrainte
password_hash	TEXT	NOT NULL
rgpd_consent_at	TIMESTAMPTZ	horodatage du consentement RGPD
total_points	INTEGER	NOT NULL, DEFAULT 0
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Table `mobility_profiles`

Colonne	Type	Contrainte
user_id	UUID	PK, FK users.id ON DELETE CASCADE
preferred_modes	TEXT[]	NOT NULL, DEFAULT {}
max_walk_minutes	INTEGER	NOT NULL, DEFAULT 15
preference	ENUM('eco', 'fast', 'balanced')	DEFAULT 'balanced'
pmr_accessibility	BOOLEAN	NOT NULL, DEFAULT false
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Table `trips` (alimente le tableau de bord citoyen et la gamification)

Colonne	Type	Contrainte
id	UUID	PK, DEFAULT gen_random_uuid()
user_id	UUID	FK users.id ON DELETE CASCADE, INDEX
origin	GEOGRAPHY(POINT, 4326)	NOT NULL, index GiST
destination	GEOGRAPHY(POINT, 4326)	NOT NULL, index GiST
modes_used	TEXT[]	NOT NULL, DEFAULT {}
primary_mode	TEXT	NOT NULL, DEFAULT 'walk', INDEX
co2_saved_grams	INTEGER	NOT NULL, DEFAULT 0
points_earned	INTEGER	NOT NULL, DEFAULT 0
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now(), INDEX

La durée totale, la distance totale et le score renvoyés par l'endpoint de calcul sont des indicateurs calculés à la volée par le moteur de scoring. Ils ne sont pas persistés : seuls les indicateurs de synthèse nécessaires au tableau de bord et à la gamification (économie CO2, points gagnés) sont conservés dans `trips`.

La rétention des enregistrements `trips` est limitée à 12 mois, conformément à la politique définie dans la section consacrée aux contraintes RGPD. Un job planifié supprime les enregistrements dont `created_at` est antérieur à cette limite.

4.3 Cas limites

Le principe de “fail fast” (échouer tôt, de manière explicite) est appliqué à chaque point d'entrée du planificateur. Chaque cas limite dispose d'un comportement défini, visible et explicite pour l'utilisateur.

Géolocalisation refusée ou indisponible. Quand l'utilisateur refuse la permission ou que le navigateur ne prend pas en charge l'API de géolocalisation, le champ “départ” reste vide et éditable. Un message invite à saisir l'adresse manuellement. Aucune logique alternative ne tente d'inférer la position. Ce comportement est conforme à l'exigence de consentement explicite définie dans la section sur les contraintes RGPD.

Timeout d'un provider externe. Chaque appel aux APIs de transport est encapsulé dans un timeout de 5 secondes. Si un provider dépasse ce délai, `routing.service` utilise les résultats du provider disponible. Si les deux providers sont en timeout, le système renvoie une erreur 503 et l'interface affiche le message “Service de calcul temporairement indisponible”. En environnement de démonstration (`DEMO_MODE=true`), ce cas ne peut pas survenir : DemoProvider répond depuis des fichiers JSON statiques sans aucune dépendance réseau.

Aucun itinéraire trouvé. Si les filtres durs éliminent tous les itinéraires candidats (aucun mode compatible, contrainte PMR trop restrictive, zone non couverte), le système renvoie une erreur 404 avec un corps JSON structuré indiquant la raison. Ce choix traite l'absence de résultat comme une ressource introuvable pour la combinaison de paramètres soumise (origine, destination, modes, contraintes), plutôt que comme une liste vide valide : la requête ne peut structurellement pas aboutir tant que l'utilisateur ne modifie pas ses critères, ce qui la distingue d'une recherche simplement infructueuse. L'interface affiche un message explicite et propose à l'utilisateur d'assouplir ses filtres.

Coordonnées hors zone nantaise. Le schéma Zod de validation inclut des contraintes de borne sur les coordonnées GPS : latitude entre 47.0 et 47.5, longitude entre -2.0 et -1.0 (emprise de Nantes Métropole). Toute requête hors de cette zone retourne une erreur 400 avec le message “Coordonnées hors de la zone de service”.

Station Bicloo trop éloignée. OsmrProvider vérifie, avant de construire un segment Bicloo, que la station la plus proche du point de départ est à moins de 1,5 km de marche. Au-delà de ce seuil, le segment Bicloo est abandonné silencieusement. L'itinéraire est construit sans ce segment ou avec un mode de remplacement si un autre provider le couvre.

4.4 Boutique de récompenses

Le module de gamification inclut une boutique de récompenses, qui permet à un citoyen d’échanger les points accumulés lors de ses trajets contre des avantages proposés par des partenaires locaux. Cette fonctionnalité répond directement à la demande de “gamification avec récompenses” formulée dans le besoin client.

Catalogue et partenaires. Chaque récompense appartient à l’un de deux types : code de réduction (`discount_code`) ou billet d’entrée (`museum_ticket`). Le périmètre MVP retient des partenaires ancrés dans la métropole nantaise : Bicloo (une heure de location offerte, 120 points), Naolib (réduction de 20 % sur un titre de transport unitaire, 150 points), Le Voyage à Nantes (Pass visite à -30 %, 400 points), ainsi que des billets d’entrée pour le Planétarium, le Muséum d’histoire naturelle, le Musée d’Arts et le Château des Ducs de Bretagne (de 500 à 900 points selon le partenaire). Une récompense désactivée (`active = false`) disparaît du catalogue proposé mais reste visible dans l’historique des échanges déjà réalisés.

Coût en points. Le coût de chaque récompense est fixé au niveau du catalogue, pas calculé dynamiquement. Un citoyen ne peut échanger une récompense que si son solde de points cumulé est supérieur ou égal au coût affiché.

Code unique de récompense. Chaque échange génère un code au format `RDM-XXXXXXXX` (8 caractères hexadécimaux dérivés d’un UUID). L’unicité est garantie par une contrainte `UNIQUE` en base ; en cas de collision, rare mais possible, le système régénère un nouveau code jusqu’à 3 tentatives avant d’échouer.

Atomicité de l’échange. Le débit du solde de points et la création de l’enregistrement d’échange s’exécutent dans une seule transaction. Le débit est conditionnel (`UPDATE ... WHERE total_points >= coût`), ce qui élimine tout risque de solde négatif en cas d’échanges simultanés sur le même compte, sans recourir à un verrou explicite.

Cas d’erreur. Trois cas sont distingués et renvoyés avec un code HTTP adapté : récompense introuvable (404), récompense désactivée ou solde insuffisant (409), fréquence d’échange excessive (429, plafond de 30 échanges par heure et par utilisateur).

4.5 Traçabilité

Le tableau suivant relie chaque user story à l’endpoint correspondant, au fichier d’implémentation principal et au test Vitest associé. Cette traçabilité garantit que chaque besoin fonctionnel est couvert à chaque couche de la solution.

User story	Endpoint	Fichier d’implémentation	Test Vitest
US-01 : calcul d’itinéraire	POST /api/routing/journeys	server/modules/routing/routing.service.ts	routing.service.test.ts : scénarios TC + modes actifs
US-02 : profil de préférence	POST /api/routing/journeys (champ <code>preference</code>)	server/modules/routing/scoring.service.ts	scoring.service.test.ts : vérification de l’ordre de tri par profil
US-03 : filtre PMR	POST /api/routing/journeys (champ <code>pmrAccessibility</code>)	routing.service.ts (filtres) + scoring.service.ts (pénalités)	routing.service.test.ts : scénarios filtre PMR et pénalités
US-04 : géolocalisation et saisie manuelle	Pas d’endpoint (logique frontend)	src/client/hooks/useGeolocation.ts + src/client/components/JourneyForm.tsx	useGeolocation.test.ts : permission accordée et refusée (mock API)
US-05 : empreinte carbone	POST /api/routing/journeys (champs <code>totalCo2g</code> , <code>co2SavedVsCarG</code>)	server/modules/routing/scoring.service.ts	scoring.service.test.ts : vérification des facteurs ADEME par mode

5. Architecture technique

5.1 Architecture globale

La solution s'articule en trois couches. Le frontend est une PWA (Progressive Web App) React servie depuis Vercel. Le backend est un serveur Express modulaire hébergé sur Render, exposant une API REST documentée. La base de données PostgreSQL avec l'extension PostGIS est hébergée sur Supabase en région eu-west-3. Quatre des cinq APIs externes (Transitous, OSRM, OpenWeatherMap, GBFS Bicloo) sont accessibles uniquement depuis le backend, jamais depuis le client : aucune clé d'API ne transite dans le navigateur, et les coordonnées GPS saisies par l'utilisateur ne sont jamais transmises directement à un tiers depuis le poste client. La cinquième, CartoDB (tuiles cartographiques), est appelée directement par Leaflet depuis le navigateur : chaque requête de tuile révèle au fournisseur la zone géographique affichée à l'écran, sans coordonnée exacte, donnée nominative ni identifiant de compte. C'est la seule exception à la règle de centralisation backend, documentée comme telle.

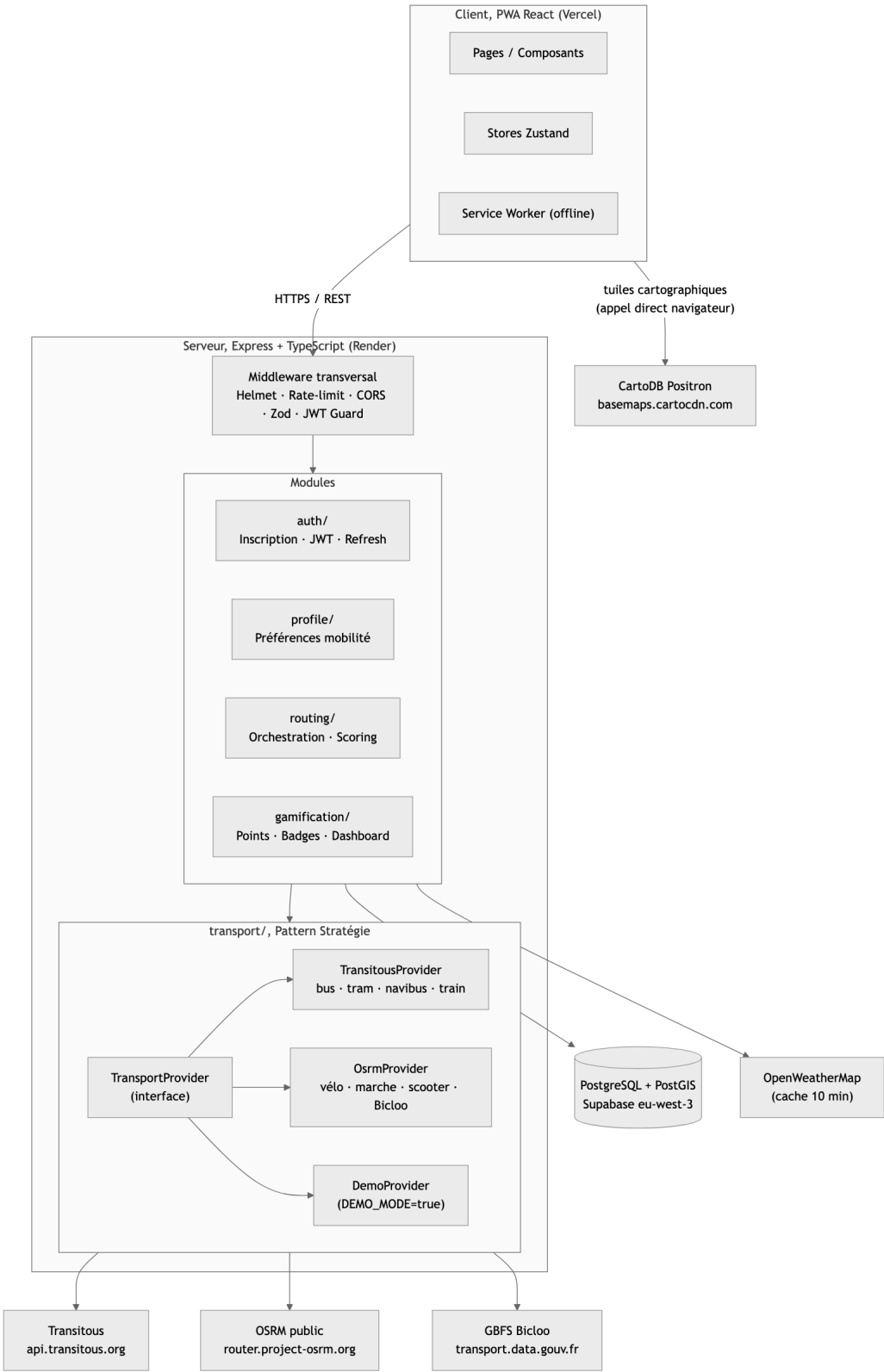


Schéma d'architecture

Le middleware transversal (Helmet, rate-limit, CORS, validation Zod, contrôle JWT) s'applique en amont de chaque handler de route. Cette centralisation garantit qu'aucune entrée utilisateur n'atteint la logique métier sans avoir été validée, et qu'aucune réponse ne franchit la couche HTTP sans les en-têtes de sécurité requis.

Le module `routing` est le seul point d'entrée vers la couche transport. Il n'appelle jamais directement Transitous ou OSRM : il délègue à l'interface `TransportProvider`, décrite dans la section suivante.

5.2 Monolithe modulaire

Comme justifié dans la section Pilotage, les microservices sont écartés au nom du principe YAGNI (You Aren't Gonna Need It). Sur un MVP développé par une seule personne en douze semaines, ils introduiraient un surcoût opérationnel sans bénéfice fonctionnel : service discovery, tracing distribué, gestion de transactions inter-services, pipelines de déploiement indépendants pour chaque service.

Ce que le monolithe modulaire garantit, que les microservices auraient garanti aussi : les frontières entre modules sont strictes. `routing` n'importe jamais depuis `gamification`. `auth` ne connaît pas `profile`. Le répertoire `server/modules/` matérialise ces frontières dans la structure même du code source.

Ce que cette discipline rend possible en V2 : l'extraction d'un module en service indépendant ne requiert pas de réécriture fonctionnelle. La séparation est effective au niveau du code ; ajouter une couche de communication HTTP ou un bus de messages entre modules est un changement de transport, pas un changement de logique.

5.3 Pattern Stratégie

Le pattern Stratégie est le point architectural central de la solution. Le module `routing` ne connaît jamais directement Transitous, OSRM ni aucun provider. Il interagit exclusivement avec l'interface `TransportProvider` définie dans `server/modules/transport/transport-provider.interface.ts` :

```
export interface TransportProvider {
  readonly supportedModes: TransportMode[]
  getJourneys(
    from: Coordinates,
    to: Coordinates,
    options: JourneyOptions
  ): Promise<Journey[]>
}
```

Trois implémentations sont prévues pour le MVP :

Implémentation	Modes couverts	Source de données
<code>TransitousProvider</code>	bus, tramway, navibus, train	api.transitous.org, format MOTIS/OTP
<code>OsrmProvider</code>	vélo, marche, scooter électrique, stations Bicloo	router.project-osrm.org (vitesses constantes par mode) + GBFS Bicloo via transport.data.gouv.fr
<code>DemoProvider</code>	tous les modes	Fichiers JSON statiques dans <code>demo-data/</code>

La fonction `selectProviders()` dans `routing.service.ts` sélectionne les providers selon les modes demandés par l'utilisateur : au moins un mode TC active `TransitousProvider`, au moins un mode actif active `OsrmProvider`. Si la variable d'environnement `DEMO_MODE=true`, seul `DemoProvider` est instancié, quelle que soit la sélection. Les appels aux providers retenus sont parallélisés via `Promise.all()`, ce qui permet d'atteindre la cible de p95 inférieur à 2 secondes documentée dans la section Pilotage.

Extension en V2 sans modification du module `routing`. Intégrer les données SIRI-Lite en temps réel, ajouter un opérateur de covoiturage ou un nouveau réseau TC se résume à écrire une classe qui implémente `TransportProvider` et à l'inclure dans la logique de `selectProviders()`. Aucune ligne du module `routing` n'est modifiée. Cette garantie d'extension sans régression est la traduction directe du principe OCP (Open/Closed Principle) : le module de routage est ouvert à l'extension, fermé à la modification.

5.4 Maintenabilité

TypeScript strict. L'option `"strict": true` active l'ensemble des contrôles statiques : vérifications de nullabilité, interdiction implicite du type `any`, inférence de retour. Toute erreur de type est bloquante à la compilation. Le type `unknown` est utilisé à la place de `any` aux frontières du système (réponses d'API externe, paramètres de routes), suivi d'un type guard explicite.

Types partagés front/back. Le répertoire `shared/types/` contient les interfaces communes aux deux parties de l'application : `Journey`, `JourneySegment`, `Coordinates`, `TransportMode`, entre autres. Un changement de contrat d'API se propage automatiquement aux deux côtés via le compilateur TypeScript. Aucune désynchronisation entre le schéma de réponse du backend et les types consommés par le frontend n'est possible sans erreur de compilation.

Séparation controller / service / middleware. Dans chaque module backend, le controller parse la requête HTTP et renvoie la réponse structurée. La logique métier est dans le service. La validation des entrées est un middleware Zod déclaré sur la route, jamais dans le controller. Cette séparation permet de tester les services indépendamment de la couche HTTP (pas de mock de `req/res`), et de réutiliser la logique de validation dans d'autres contextes.

Migrations SQL versionnées. Les évolutions de schéma sont stockées dans `server/db/migrations/` avec un préfixe numérique séquentiel : `001-create-users.sql` , `002-create-profiles.sql` . L'ordre d'application est déterministe et reproductible sur tous les environnements (développement local, intégration, production Supabase).

Conventions de nommage. Les fichiers sont en kebab-case (`routing.service.ts` , `journey-card.tsx`). Les composants React sont en PascalCase. Les variables d'environnement sont en SCREAMING_SNAKE_CASE. Ces conventions sont appliquées via la configuration ESLint et TypeScript et constituent le contrat implicite qui facilite l'intégration d'un nouveau développeur.

5.5 Interopérabilité

API REST documentée via Swagger. Le backend expose une API REST dont le contrat est documenté via Swagger (OpenAPI 3.0), accessible à l'adresse `/api/docs` . La solution ne se contente pas de consommer des APIs externes : elle en expose une. Cette API ouvre la solution vers les systèmes d'information tiers de Nantes Métropole (portail citoyen, outils de pilotage des mobilités) et permet une intégration sans accès au code source.

Standards de transport consommés et respectés

Standard	Rôle	Usage dans la solution
GTFS (General Transit Feed Specification)	Format ouvert de données TC statiques : horaires, arrêts, lignes	Transitous agrège les flux GTFS Naolib et les expose via l'API MOTIS/OTP consommée par <code>TransitousProvider</code>
GBFS (General Bikeshare Feed Specification)	Standard ouvert pour les vélos en libre-service	Flux Bicloo via <code>transport.data.gouv.fr</code> , consommé par <code>OsrnProvider</code> pour localiser les stations
SIRI-Lite (Service Interface for Real-time Information)	Standard européen d'échanges temps réel entre SI de transport	Prochains passages Naolib ; intégration prévue en V2, hors périmètre MVP
MOTIS/OTP	Format de réponse de l'API Transitous (open source routing)	Réponse parsée par <code>TransitousProvider</code> et transformée en type <code>Journey</code> interne
GeoJSON	Format ouvert d'échange de données géographiques	Géométrie des segments d'itinéraire (LineString) dans la réponse de l'API

Tous les formats de données internes reposent sur des structures JSON standard sans dépendance à un format propriétaire. La portabilité de la solution vers une autre collectivité française disposant d'un flux GTFS ouvert est immédiate : la configuration de l'URL Transitous et de la zone de service géographique suffit.

5.6 Modèle de données

Le schéma PostgreSQL couvre quatre modules fonctionnels : authentification, profil de mobilité, historique de trajets et gamification (badges et récompenses). Les tables `users` , `mobility_profiles` et `trips` , décrites en détail dans la section consacrée aux spécifications du planificateur multimodal, constituent le socle du modèle. Les tables suivantes complètent le schéma pour la gamification et la boutique de récompenses.

Table `badges`

Colonne	Type	Contrainte
id	UUID	PK, DEFAULT <code>gen_random_uuid()</code>
name	TEXT	UNIQUE, NOT NULL
description	TEXT	NOT NULL
threshold_type	ENUM('total_trips', 'co2_saved', 'total_points', 'streak_days')	NOT NULL
threshold_value	INTEGER	NOT NULL

Table `user_badges` (clé primaire composée)

Colonne	Type	Contrainte
user_id	UUID	PK, FK <code>users.id</code> ON DELETE CASCADE, INDEX
badge_id	UUID	PK, FK <code>badges.id</code> ON DELETE CASCADE
unlocked_at	TIMESTAMPTZ	NOT NULL, DEFAULT <code>now()</code>

Table `rewards`

Colonne	Type	Contrainte
id	UUID	PK, DEFAULT <code>gen_random_uuid()</code>
name	TEXT	UNIQUE, NOT NULL
description	TEXT	NOT NULL

Colonne	Type	Contrainte
reward_type	ENUM('discount_code', 'museum_ticket')	NOT NULL
points_cost	INTEGER	NOT NULL, CHECK > 0
partner_name	TEXT	NOT NULL
active	BOOLEAN	NOT NULL, DEFAULT true
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Table reward_redemptions

Colonne	Type	Contrainte
id	UUID	PK, DEFAULT gen_random_uuid()
user_id	UUID	FK users.id ON DELETE CASCADE, INDEX
reward_id	UUID	FK rewards.id ON DELETE RESTRICT
code	TEXT	UNIQUE, NOT NULL
points_spent	INTEGER	NOT NULL
redeemed_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Les points gagnés à chaque trajet sont crédités directement sur `trips.points_earned` et cumulés sur `users.total_points`. Aucune table de journal séparée n'est nécessaire : le solde de points d'un utilisateur se déduit de `users.total_points`, et son historique de gain se retrouve trajet par trajet dans `trips`.

Index PostGIS

Les colonnes `origin` et `destination` de la table `trips` utilisent le type `GEOGRAPHY(POINT, 4326)` de l'extension PostGIS (Geographic Information System, système d'information géographique, avec projection WGS84). Un index spatial de type GiST (Generalized Search Tree) est prévu sur chacune de ces colonnes pour accélérer les requêtes géospatiales (calcul de distance, sélection d'itinéraires dans un rayon donné) :

```
CREATE INDEX idx_trips_origin      ON trips USING GIST(origin);
CREATE INDEX idx_trips_destination ON trips USING GIST(destination);
```

Politique de rétention

Les enregistrements de `trips` seront supprimés automatiquement après douze mois, conformément à la politique RGPD définie dans la section consacrée aux contraintes transversales. Un job planifié côté backend exécute cette purge en dehors des heures de pointe. Aucune donnée GPS d'un utilisateur n'est conservée au-delà de cette limite sans action volontaire de sa part.

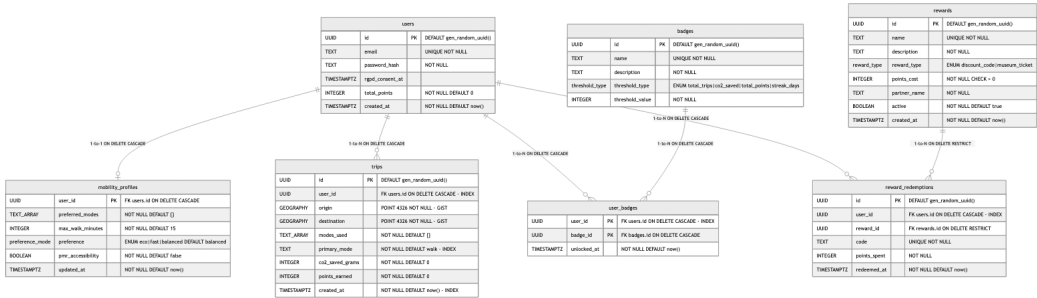


Schéma relationnel complet

Index

Table	Colonne(s)	Type	Migration
users	email	B-tree UNIQUE	002
trips	user_id	B-tree	004
trips	origin	GiST PostGIS	004
trips	destination	GiST PostGIS	004
trips	created_at	B-tree	004
trips	(user_id, primary_mode, created_at)	B-tree composé	012
user_badges	user_id	B-tree	006
reward_redemptions	user_id	B-tree	013

Description du modèle

Le schéma s'organise autour de quatre entités racines. `users` porte l'identité et le solde de points cumulé (`total_points`), dénormalisé pour éviter un recalcul par agrégation à chaque affichage du tableau de bord. `mobility_profiles` est en relation 1-1 avec `users` : chaque utilisateur possède exactement un profil de mobilité, supprimé en cascade avec son compte.

`trips` enregistre chaque itinéraire effectué, avec les coordonnées d'origine et de destination en `GEOGRAPHY (POINT, 4326)` pour permettre des requêtes géospatiales (distance, recherche par rayon) via les index GiST. Seuls les indicateurs de synthèse utiles à la gamification et au tableau de bord sont persistés (`co2_saved_grams`, `points_earned`) : la durée, la distance et le score détaillés d'un itinéraire restent des valeurs calculées à la volée par le moteur de scoring, jamais stockées.

`badges` et `user_badges` modélisent la progression de gamification : `badges` définit un catalogue de paliers (nombre de trajets, CO2 économisé, points totaux, séquence de jours consécutifs) et `user_badges` matérialise le déverrouillage d'un badge par un utilisateur, avec une clé primaire composée (`user_id`, `badge_id`) qui interdit nativement le déverrouillage en double.

`rewards` et `reward_redemptions` modélisent la boutique de récompenses : `rewards` catalogue les récompenses disponibles auprès de partenaires (coût en points, type), et `reward_redemptions` trace chaque échange avec un code unique généré à l'échange. La suppression d'une récompense est bloquée (`ON DELETE RESTRICT`) tant que des échanges y font référence, ce qui préserve l'intégrité de l'historique.

6. Contraintes transversales

Les douze contraintes officielles du projet traversent l'ensemble de la solution sans se cantonner à un seul module fonctionnel. Cette section les traite comme un référentiel de conformité : pour chaque contrainte, l'équipe indique ce qui est prévu au périmètre, pourquoi l'approche retenue est la plus appropriée, et quelle preuve vérifiable en atteste.

Tableau de synthèse

#	Contrainte	Réponse au périmètre MVP	Preuve
C1	PWA installable	<code>vite-plugin-pwa</code> : manifest + service worker, installation depuis le navigateur	Capture d'écran de l'invite d'installation
C2	Responsive et UX	Tailwind mobile-first, design system UrbanFlow (mode clair et sombre), 5 écrans validés en maquette	Maquettes Figma
C3	Normes et standards	TypeScript strict, ESLint, conventions REST, standards transport GTFS/GBFS	Extrait config ESLint + Swagger
C4	Sécurité OWASP	Helmet, rate-limit, CORS, Zod, bcrypt, JWT HttpOnly	Extrait <code>server/index.ts</code> (empilement middleware)
C5	Éco-conception	Bundle < 300 ko gzip, lazy loading des routes, Leaflet 40 ko, cache météo 10 min	Rapport <code>vite-bundle-visualizer</code>
C6	Géolocalisation	Consentement explicite, géoloc navigateur, fallback saisie manuelle	Capture écran consentement
C7	Accessibilité WCAG 2.1 AA	Contrastes >= 4.5:1, navigation clavier, <code>alt</code> , <code>aria-label</code> , <code>role=application</code>	Audit Axe DevTools : 0 erreur bloquante
C8	RGPD	Consentement géoloc, <code>DELETE /api/users/me</code> , rétention 12 mois, zéro partage GPS	Capture écran consentement + extrait endpoint
C9	Interopérabilité	API REST OpenAPI 3.0 (Swagger), pattern Stratégie (TransportProvider), GTFS/GBFS/GeoJSON	Page <code>/api/docs</code>
C10	Performances mobilité	Cache Zustand, service worker offline, <code>Promise.all()</code> , Lighthouse > 90 en performance	Rapport Lighthouse
C11	Sécurité des données de déplacement	HTTPS obligatoire, authentification JWT sur toutes les routes de données, minimisation	Config Helmet + Render HTTPS forcé
C12	Normes accessibilité transport	Filtre PMR, seuil marche 5 min, pénalité vélo PMR (conformité des arrêts : reportée en V2)	Extrait <code>scoring.service.ts</code>

6.1 PWA installable

La contrainte impose une application installable sur terminal mobile sans passer par un store. La solution utilise `vite-plugin-pwa`, qui génère automatiquement le fichier manifest (nom, icônes, couleur de thème, mode d'affichage `standalone`) et le service worker au moment du build.

L'installation est proposée par le navigateur via une bannière native déclenchée par le manifest. Aucune démarche supplémentaire n'est requise de la part de l'utilisateur sur Android. Sur iOS Safari, la procédure est guidée par un message contextuel affiché une seule fois.

Le service worker met en cache l'app shell (HTML, CSS, JavaScript, polices) et les tuiles cartographiques déjà consultées. La stratégie retenue est "stale-while-revalidate" pour l'app shell : l'interface se charge depuis le cache sans attendre le réseau, puis se met à jour silencieusement. Le calcul d'un nouvel itinéraire reste dépendant des APIs de routage : ce cas est hors périmètre offline en MVP et documenté comme tel dans la section Contexte.

6.2 Responsive et expérience utilisateur

TailwindCSS impose une discipline mobile-first : toute classe sans préfixe s'applique au mobile, les préfixes `md:` et `lg:` étendent la mise en page aux écrans plus larges. Aucune mise en page n'est conçue pour desktop puis adaptée : le point de départ est systématiquement le viewport mobile (375 px).

Le design system UrbanFlow définit les tokens de couleur pour un mode clair et un mode sombre, ainsi que les composants réutilisables (boutons, cartes d'itinéraire, bottom navigation). Ces tokens sont déclarés dans `@theme` de `src/client/index.css` et génèrent automatiquement les classes Tailwind correspondantes. La cohérence visuelle est ainsi garantie structurellement, sans duplication de valeurs hexadécimales dans les composants.

Cinq écrans constituent le parcours principal validé en maquette : carte (page d'accueil), planificateur (formulaire de recherche), résultats, profil utilisateur et tableau de bord de mobilité.

6.3 Normes et standards

TypeScript strict. L'option `"strict": true` dans `tsconfig.json` active la totalité des vérifications statiques : nullabilité, types de retour inférés, interdiction implicite de `any`. Toute erreur de type est bloquante à la compilation, non au runtime.

ESLint. Un fichier de configuration `eslint.config.js` (format flat config ESLint 9) applique les règles recommandées de `typescript-eslint`, avec des surcouches strictes propres au projet (`@typescript-eslint/no-explicit-any` et `@typescript-eslint/consistent-type-imports` en erreur bloquante sur le client, le serveur et le code partagé). Le pipeline GitHub Actions exécute ce lint à chaque pull request ; aucun merge n'est autorisé avec des avertissements actifs sur les fichiers modifiés.

Conventions REST. Les routes de l'API respectent les conventions REST : noms de ressources au pluriel, verbes HTTP sémantiques (`POST` pour la création, `DELETE` pour la suppression), codes de statut HTTP normalisés. Le contrat de l'API est documenté en OpenAPI 3.0 via Swagger (voir C9).

Standards transport. La solution consomme et respecte les standards ouverts du secteur : GTFS (General Transit Feed Specification, format ouvert de données TC statiques) via Transitous, GBFS (General Bikeshare Feed Specification, standard vélos en libre-service) pour les stations Bicloo, et GeoJSON (format ouvert d'échange de données géographiques) pour la géométrie des segments. Ces standards sont détaillés dans la section Architecture.

6.4 Sécurité OWASP

La couche de sécurité est organisée en middleware transversal, appliqué en amont de toutes les routes sans exception.

Helmet. Le middleware Helmet positionne les en-têtes HTTP de sécurité sur chaque réponse : `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Strict-Transport-Security`. Ces en-têtes couvrent les vecteurs d'attaque XSS (Cross-Site Scripting) par injection de scripts tiers et de clickjacking.

Rate limiting. Un limiteur de débit (100 requêtes par fenêtre de 15 minutes par adresse IP) protège les endpoints d'authentification contre les attaques par force brute. La configuration est plus restrictive sur les routes `/api/auth/login` et `/api/auth/register` que sur les routes de données.

CORS. La politique d'origine croisée n'autorise que l'origine du frontend (`CORS_ORIGIN` configuré en variable d'environnement). Le middleware `cors` n'émet pas de code d'erreur : une origine non autorisée ne reçoit simplement pas les en-têtes `Access-Control-Allow-*` sur la réponse, ce qui empêche un navigateur tiers d'exploiter cette réponse depuis une page web. Un client non-navigateur (script, `curl`) reçoit la réponse normalement, car CORS est une protection appliquée par le navigateur, pas par le serveur. La protection réelle des données repose sur l'authentification JWT présentée en fin de section, pas sur CORS seul.

Validation Zod. Toutes les entrées utilisateur (corps de requête, paramètres de route) passent par un middleware Zod avant d'atteindre le controller. La validation retourne un code 400 structuré (`{ error: string, details: ZodError }`) en cas d'entrée invalide. Aucune entrée non validée n'atteint la logique métier.

Authentification et hashage. Les mots de passe sont hachés avec `bcrypt` (minimum 10 rounds). Les tokens JWT (JSON Web Token) d'accès ont une durée de vie de 15 minutes. Le refresh token est transmis uniquement via un cookie `HttpOnly`, `Secure` et `SameSite=Strict`, ce qui protège contre les attaques XSS (le JavaScript de la page ne peut pas lire le cookie) et CSRF (Cross-Site Request Forgery). Aucun token n'est stocké dans `localStorage`.

L'empilement est déclaré en tête de `src/server/index.ts`, avant le montage de toute route applicative :

```
app.set('trust proxy', 1)

const globalRateLimit = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 100,
  standardHeaders: 'draft-8',
  legacyHeaders: false,
  message: { error: 'Trop de requêtes, réessayez plus tard' },
})

app.use(helmet())
app.use(cors({ origin: process.env.CORS_ORIGIN, credentials: true }))
app.use(globalRateLimit)
app.use(express.json())
app.use(cookieParser())
```

6.5 Éco-conception

L'éco-conception est une contrainte de conception, pas un ajout a posteriori. Elle se traduit par quatre règles appliquées à chaque décision technique.

Bundle frontend inférieur à 300 ko gzip. Mesure sur le build de référence (`npm run build`) : le bundle JavaScript et CSS complet totalise environ 247 ko gzip, sous la cible de 300 ko. Le poste le plus lourd est le chunk `vendor` (React 18, React Router, Zustand : 134 ko gzip), suivi du chunk cartographique Leaflet (43 ko gzip) et de `MapPage` (18 ko gzip, page la plus dense de l'application). Les graphiques du tableau de bord (répartition CO2 hebdomadaire, répartition par mode) sont implémentés en SVG natif plutôt qu'avec une bibliothèque de charting tierce, ce qui évite d'ajouter Recharts ou équivalent au bundle. Les bibliothèques volumineuses sans alternative légère équivalente sont exclues du périmètre (Google Maps SDK : plus de 200 ko).

Lazy loading des routes. Chaque page React est chargée via `React.lazy` et `Suspense`. L'utilisateur ne télécharge que le code de la page sur laquelle il se trouve. Les pages peu fréquentées (tableau de bord, paramètres) ne sont chargées qu'à la première navigation.

Cache météo. Les données OpenWeatherMap sont mises en cache en mémoire côté serveur (Map JavaScript, TTL de 10 minutes) pour éviter les appels redondants sur des requêtes portant sur le même secteur géographique dans la même fenêtre de temps. Ce cache réduit le nombre d'appels réseau et la latence de scoring, comme décrit dans la section Pilotage.

Réduction des images et ressources. Les images de l'interface sont servies au format WebP lorsque possible. Les icônes de transport sont des pictogrammes SVG inline (zéro requête HTTP supplémentaire).

6.6 Géolocalisation

La géolocalisation du navigateur est utilisée uniquement pour préremplir le champ "départ" dans le planificateur. Elle n'est jamais activée en arrière-plan ni conservée côté serveur au-delà de la durée de la requête de calcul.

Consentement explicite. Avant toute activation de l'API de géolocalisation, une modale d'information s'affiche : elle explique l'usage prévu (préremplir l'adresse de départ), la durée de traitement (non conservé), et présente deux options explicites ("Autoriser pour ce trajet" et "Saisir manuellement"). Le refus n'entraîne aucune dégradation fonctionnelle : le champ reste éditable. Ce comportement est lié à la contrainte C8 et détaillé dans la section consacrée aux spécifications du planificateur multimodal.

Fallback saisie manuelle. Si l'utilisateur refuse la géolocalisation, si le navigateur ne prend pas en charge l'API, ou si la précision GPS est insuffisante (contexte indoor, tunnel), le champ "départ" reste vide et éditable. L'interface n'infère pas la position par d'autres moyens (adresse IP, données réseau) : cela constituerait un traitement de données de localisation sans consentement explicite.

6.7 Accessibilité WCAG 2.1 AA

L'accessibilité est vérifiée sur les cinq écrans du parcours principal avec Axe DevTools (extension Chrome), outil de référence pour l'audit WCAG (Web Content Accessibility Guidelines) 2.1 niveau AA. L'objectif est zéro erreur bloquante détectée par Axe sur l'ensemble de ces écrans.

Contrastes. La palette UrbanFlow est déclinée en mode clair et en mode sombre, chacun construit pour que le ratio de contraste entre le texte et le fond dépasse 7:1 pour le texte principal et 4.5:1 pour le texte secondaire, dans les deux modes. Ces valeurs dépassent le seuil WCAG AA (4.5:1 pour le texte normal).

Navigation clavier. Chaque élément interactif (boutons, liens, champs de formulaire, options de modes de transport) est atteignable et activable au clavier dans un ordre logique. Le focus visible est explicitement stylisé via les classes Tailwind `focus:ring` pour ne jamais être masqué.

Attributs ARIA et alternatives textuelles. Tout bouton sans libellé textuel visible porte un attribut `aria-label`. Chaque icône de mode de transport est accompagnée d'un texte alternatif ou d'un span `sr-only` (screen-reader only) lisible par les lecteurs d'écran. La carte Leaflet porte les attributs `role="application"` et `aria-label="Carte des itinéraires Nantes Métropole"` pour signaler aux technologies d'assistance qu'il s'agit d'une zone interactive non standard. Chaque champ de formulaire est associé à son label via `htmlFor` et `id`.

6.8 RGPD

Le traitement des données personnelles suit les principes de minimisation et de transparence du RGPD (Règlement Général sur la Protection des Données).

Consentement géolocalisation. Le mécanisme est décrit en C6. Aucune position GPS n'est collectée ni transmise au backend sans que l'utilisateur ait activement accepté pour la session en cours.

Droit à l'effacement. L'endpoint `DELETE /api/users/me` supprime en cascade l'ensemble des enregistrements liés au compte : profil, historique de trajets, points, badges et récompenses échangées. La suppression est irréversible et effective immédiatement en base de production. Les sauvegardes techniques de l'hébergeur restent un cas distinct : Supabase ne réalise pas de sauvegarde ciblée par utilisateur mais des instantanés de l'ensemble de la base, dont la rétention suit la politique de l'hébergeur et n'est pas pilotable par une requête applicative. Cette rétention technique globale, de l'ordre de quelques jours selon le plan souscrit, correspond à la tolérance admise par la CNIL (Commission Nationale de l'Informatique et des Libertés) pour les sauvegardes de sécurité : l'obligation d'effacement porte sur les traitements actifs, pas sur les copies de sécurité destinées à la seule reprise après incident.

Durée de conservation. Les enregistrements de la table `trips` sont supprimés automatiquement après 12 mois. Un job planifié exécute cette purge en dehors des heures de pointe. La durée de 12 mois est justifiée par la nécessité de calculer les statistiques

annuelles de mobilité affichées dans le tableau de bord citoyen.

Zéro partage GPS. Les coordonnées de départ et d'arrivée d'un trajet sont transmises du client vers le backend uniquement pour le calcul de l'itinéraire. Elles ne sont pas transmises aux APIs de transport dans leur forme brute : Transitous et OSRM reçoivent des coordonnées arrondies à 4 décimales (précision de l'ordre de 10 mètres). Aucune donnée de géolocalisation n'est partagée avec des tiers à des fins commerciales ou analytiques. Cette règle porte sur les coordonnées de recherche transmises au backend ; elle est distincte du chargement des tuiles cartographiques CartoDB par le navigateur, appel client direct documenté comme exception dans la section consacrée à l'architecture technique.

6.9 Interopérabilité

La solution ne se contente pas de consommer des APIs : elle expose la sienne. Cette double posture est la traduction technique de la contrainte d'interopérabilité.

API REST documentée via Swagger. Le backend expose l'ensemble de ses endpoints via une page OpenAPI 3.0 accessible à `/api/docs`. Cette documentation est générée automatiquement depuis les annotations `swagger-jsdoc` présentes dans le code source. Elle décrit les schémas de requête, les schémas de réponse et les codes d'erreur pour chaque endpoint. Les systèmes d'information tiers de Nantes Métropole peuvent consommer l'API sans accès au code source.

Pattern Stratégie. L'interface `TransportProvider` est elle-même un contrat d'interopérabilité interne : tout provider de transport conforme à cette interface est immédiatement intégrable sans modifier le module de routage. Ce mécanisme est détaillé dans la section Architecture.

Standards de données de transport. La solution s'appuie sur les standards ouverts du secteur transport (GTFS, GBFS, SIRI-Lite, GeoJSON), tous définis dans la section Architecture. Ces standards garantissent la portabilité de la solution vers d'autres collectivités disposant d'un flux GTFS ou GBFS ouvert.

6.10 Performances mobilité

Les performances en contexte de mobilité sont contraintes par deux réalités : la connectivité réseau variable et la puissance limitée des terminaux d'entrée de gamme.

Parallélisation des appels. Comme décrit dans la section Pilotage, `routing.service.ts` lance les appels à `TransitousProvider` et `OsrmProvider` simultanément via `Promise.all()`. Cette mesure ramène le p95 de calcul d'itinéraire sous la cible de 2 secondes, contre un p95 estimé à plus de 3 secondes en appels séquentiels.

Cache Zustand. Les données déjà chargées (profil utilisateur, résultats de la dernière recherche, badges) sont conservées dans le store Zustand côté client. Un composant qui accède à ces données ne déclenche pas de nouvel appel réseau si les données sont déjà présentes. La durée de vie du cache Zustand correspond à la session de navigation.

Service worker offline. L'app shell est servi depuis le cache du service worker, ce qui rend la première interaction visible en moins de 1 seconde même sur réseau 3G. Les tuiles cartographiques déjà consultées sont également mises en cache, permettant à l'utilisateur de consulter la carte d'un trajet déjà calculé sans connexion.

Cible Lighthouse. Le score de performance Lighthouse cible est supérieur à 90 sur mobile (simulation de réseau 4G lente, CPU x4). Ce seuil couvre les terminaux d'entrée de gamme qui constituent une fraction significative de la base utilisateurs visée.

6.11 Sécurité des données de déplacement

Les données de déplacement (coordonnées GPS, historique de trajets) sont des données personnelles sensibles. La sécurité de leur traitement repose sur trois principes.

Chiffrement en transit. Toutes les communications entre le client et le backend transitent exclusivement en HTTPS. Le certificat TLS est géré automatiquement par Render (backend) et Vercel (frontend). Helmet positionne l'en-tête `Strict-Transport-Security` pour forcer le navigateur à rejeter toute connexion HTTP non sécurisée, même si l'URL est saisie sans le préfixe `https://`.

Accès authentifié. Chaque endpoint exposant ou acceptant des données de déplacement est protégé par le middleware JWT Guard. Un token d'accès valide (15 minutes de durée de vie) est requis pour accéder à `POST /api/routing/journeys`, `GET /api/journeys/history` et `DELETE /api/users/me`. Sans token, la réponse est un code 401 sans contenu applicatif.

Minimisation. Les coordonnées GPS transmises aux providers de transport sont arrondies à 4 décimales. Les historiques de trajets ne conservent pas la géométrie détaillée des segments (liste de points du tracé) : seuls le point de départ, le point d'arrivée, la durée, la distance et le bilan CO2 sont persistés. La géométrie ne sert qu'à l'affichage immédiat et n'est pas stockée en base.

6.12 Normes d'accessibilité transport

Cette contrainte couvre l'accessibilité des personnes à mobilité réduite (PMR) dans le contexte spécifique du transport, au-delà de l'accessibilité numérique traitée en C7.

Filtre PMR dans le profil. L'utilisateur peut activer le mode PMR dans son profil de mobilité. Ce paramètre est persisté dans la table `mobility_profiles` (colonne `pmr_accessibility`) et transmis à chaque requête de calcul d'itinéraire via le champ `pmrAccessibility` du corps de requête.

Seuil de marche réduit. Quand le filtre PMR est actif, le seuil de tolérance pour les segments de marche est réduit à la valeur minimale entre la valeur configurée par l'utilisateur et 5 minutes. Tout itinéraire comportant un segment de marche supérieur à ce seuil est éliminé avant le calcul du score.

Pénalités spécifiques dans le score de confort. Le moteur de scoring applique des pénalités renforcées en mode PMR : 60 points de pénalité sur le score de confort pour un segment de marche dépassant le seuil (contre 40 en mode standard), et 50 points de pénalité supplémentaires si un segment vélo est présent dans l'itinéraire. Ces pénalités sont décrites en détail dans la section consacrée aux spécifications du planificateur multimodal.

Compatibilité arrêts accessibles. La couverture des données d'accessibilité des arrêts Naolib dépend des flux GTFS fournis par Semitan. La conformité des arrêts (quais surélevés, annonces sonores) n'est pas vérifiable directement depuis les données ouvertes disponibles en MVP. Ce point est documenté comme un axe d'amélioration V2, conditionné à un accord opérationnel avec Semitan pour l'accès aux données d'accessibilité des arrêts.

Le seuil de marche réduit est appliqué en filtre dur dans `routing.service.ts`, avant le scoring :

```
// Filtre dur maxWalkMinutes : éliminer tout itinéraire dont un segment marche
// dépasse le seuil de l'utilisateur (PMR réduit ce seuil à 5 min).
const maxWalk = options.pmrAccessibility
  ? Math.min(options.maxWalkMinutes ?? 30, 5)
  : (options.maxWalkMinutes ?? 30)

const withWalkFilter = filtered.filter((j) =>
  j.segments.filter((s) => s.mode === 'walk').every((s) => s.durationMin <= maxWalk)
)
```

Les pénalités renforcées sont appliquées dans `computeComfortScore()` de `scoring.service.ts` :

```
// Pénalité si un segment marche dépasse le seuil
if (maxWalkSeg > maxWalk) {
  // Pénalité plus sévère si PMR (-60) que pour un utilisateur standard (-40)
  base = Math.max(0, base - (pmr ? 60 : 40))
}

// PMR : pénalité supplémentaire si le trajet contient du vélo
if (pmr && segments.some((s) => s.mode === 'bike')) {
  base = Math.max(0, base - 50)
}
```


7. Qualité, tests et gestion des bogues

7.1 Stratégie de tests

Le moteur de scoring multicritères est un algorithme déterministe : toute modification des pondérations ou des filtres PMR peut produire un résultat incorrect sans déclencher d'erreur runtime. Les tests unitaires sont la seule protection contre cette catégorie de bogues silencieux. L'équipe retient une stratégie en trois niveaux : tests unitaires sur la logique métier, tests d'intégration sur les endpoints Express, et recette manuelle sur le prototype complet.

Tests unitaires avec Vitest

Vitest est retenu face à Jest pour trois raisons : support natif des modules ES (ECMAScript Modules) sans transpilation supplémentaire, intégration TypeScript sans configuration Babel, et performances supérieures sur les suites courtes grâce au worker pool Vite. L'API est compatible Jest ; une migration éventuelle ne représente pas de charge de réécriture.

Le périmètre des tests unitaires couvre les modules à logique calculatoire pure, là où une erreur de logique ne produit pas de crash mais un résultat incorrect.

Module	Cas couverts
<code>scoring.service.ts</code>	Score final par pondération eco / rapide / équilibré ; pénalités PMR (seuil 5 min, pénalité vélo) ; calcul CO2 par segment (facteurs Base Empreinte ADEME) ; valeurs limites (score à 0, distance nulle, itinéraire mono-mode)
<code>selectProviders()</code> dans <code>routing.service.ts</code>	Sélection de <code>TransitousProvider</code> sur mode TC ; <code>OsrmProvider</code> sur mode actif ; <code>DemoProvider</code> sur <code>DEMO_MODE=true</code> ; combinaison de modes
<code>OsrmProvider</code>	Refus de construction d'un trajet Bicloo si la station la plus proche dépasse 1,5 km de marche
Constantes CO2	Vérification que les valeurs correspondent aux coefficients ADEME déclarés dans <code>shared/constants/co2-factors.ts</code>

Tests d'intégration

Les endpoints Express sont testés via Supertest (bibliothèque de test HTTP pour Node.js qui exécute les requêtes sans démarrer de serveur réseau réel) avec la variable `DEMO_MODE=true` active. Ce mode bascule tous les appels aux APIs externes vers les fichiers JSON statiques de `demo-data/`, rendant les tests d'intégration reproductibles sans dépendance réseau ni clé API.

Endpoint	Cas couverts
<code>POST /api/auth/register</code>	Inscription valide (201) ; email déjà utilisé (409) ; payload invalide (400 Zod)
<code>POST /api/auth/login</code>	Connexion valide (200 + cookie refresh) ; mauvais mot de passe (401) ; payload invalide (400)
<code>POST /api/routing/journeys</code>	Calcul nominal (200, liste non vide) ; payload sans coordonnées (400) ; token manquant (401)
<code>GET /api/journeys/history</code>	Historique avec JWT valide (200) ; sans JWT (401)
<code>DELETE /api/users/me</code>	Suppression en cascade (204) ; vérification que le compte n'est plus accessible (404 sur requête suivante)

Ce qui n'est pas automatisé en intégration continue

Deux catégories de tests restent manuels et hors du pipeline GitHub Actions.

L'audit d'accessibilité Axe DevTools est exécuté manuellement sur les cinq écrans du parcours principal avant chaque merge significatif. L'automatisation dans la CI produirait des résultats instables : JSDOM (DOM virtuel, environnement JavaScript simulé sans navigateur réel) ne reproduit pas fidèlement le rendu CSS et les interactions ARIA d'un navigateur. La procédure manuelle, détaillée dans la section consacrée à la contrainte C7, reste la référence pour l'audit de conformité WCAG.

L'audit Lighthouse est exclu de la CI GitHub Actions. Les runners partagés GitHub produisent des résultats de performance inconsistants (variance de 15 à 20 points sur des runs identiques) liée aux différences de charge CPU entre les machines virtuelles. Lighthouse est exécuté depuis Chrome DevTools en local, sur réseau simulé 4G lente et CPU x4, et le rapport est capturé comme preuve annexe.

Résultat de la suite au dernier run (`npm run test:coverage`) : 25 fichiers de test, 444 tests, tous passants. La couverture v8 s'établit à 82,8 % des instructions et 83,4 % des fonctions sur l'ensemble du code serveur, avec une couverture proche de 100 % sur les modules à logique métier critique (`rewards.service.ts` : 100 %, `auth.service.ts` : 95,9 %, `gamification.service.ts` : 87,4 %). Le module `demo-config.ts` reste sous-couvert (39 %) : il s'agit de configuration de scénarios de démonstration, à faible risque de régression fonctionnelle.

7.2 Workflow bogues

La préproduction correspond à l'environnement de déploiement continu activé dès le sprint 2 : branche `main` déployée automatiquement sur Vercel (frontend) et Render (backend) à chaque merge. Tout bogue identifié dans cet environnement suit un processus structuré.

Signalement et triage

Les bogues sont signalés via GitHub Issues avec un template de rapport imposant quatre champs : description du comportement observé, comportement attendu, étapes de reproduction, environnement (navigateur, système d'exploitation, type de réseau) et capture d'écran si applicable. Ce format standardisé réduit le délai de diagnostic en évitant les rapports incomplets qui nécessitent des allers-retours.

À l'ouverture, chaque issue reçoit un label de criticité parmi trois niveaux.

Criticité	Définition	Délai cible
Bloquant	Fonctionnalité du périmètre MVP inutilisable, faille de sécurité, crash applicatif non récupéré	Correction avant tout merge suivant
Majeur	Résultat incorrect sans crash, dégradation UX significative, test d'intégration en échec	Traitement dans le sprint courant
Mineur	Problème visuel sans impact fonctionnel, libellé incorrect, incohérence de style	Backlog, traitement si la capacité du sprint le permet

Cycle de correction

Un bogue bloquant ou majeur déclenche l'ouverture d'une branche `fix/nom-du-bug` depuis `main`. La correction est développée sur cette branche et couverte par au moins un test qui reproduit le comportement fautif avant de valider le comportement corrigé. Si le bogue ne peut pas être couvert par un test automatisé (par exemple un problème de rendu CSS), la recette manuelle est documentée dans la pull request.

La pull request cible `main` directement. Elle décrit le comportement fautif, la cause racine identifiée et la mesure corrective. Le pipeline CI (lint + build TypeScript) doit passer sans avertissement. Après merge, la correction est vérifiée dans l'environnement de préproduction avant fermeture de l'issue.

Définition de terminé pour un bogue

Un bogue est considéré terminé quand les quatre conditions suivantes sont réunies : la correction est présente dans `main` ; au moins un test automatisé reproduit puis valide le comportement attendu (ou la recette manuelle est documentée dans la PR) ; le pipeline CI est au vert ; la vérification en préproduction confirme l'absence de régression sur les écrans adjacents.

7.3 Philosophie de correction

La règle qui gouverne la correction des bogues dans ce projet est la suivante : toute correction identifie et supprime la cause racine. Un contournement n'est jamais une correction.

Cette distinction a des conséquences concrètes sur la qualité du code. Dans le planificateur multimodal, le composant de recherche d'itinéraire peut déclencher un double appel API à chaque montage : c'est le comportement attendu de React 18 en mode strict (Strict Mode), qui simule le cycle montage-démontage-remontage précisément pour exposer les effets de bord non nettoyés. La réponse instinctive serait d'introduire un flag booléen déclaré en dehors du composant pour bloquer le second appel. Cette approche résout le symptôme visible mais masque la cause réelle, introduit un état global non prévisible et crée de la dette technique.

La correction retenue utilise `AbortController` (interface Web standard permettant d'annuler une requête en cours) et la fonction de nettoyage du `useEffect` : le premier appel est annulé proprement par le `cleanup` avant que le second se déclenche. Le composant gère son propre cycle de vie sans dépendre d'un état externe. La correction améliore la robustesse dans tous les contextes, pas seulement en mode strict.

Ce principe s'étend à l'ensemble de la base de code. Un `eslint-disable` ciblé masque une erreur de type sans la corriger. Une annotation `any` contourne le système de types sans résoudre l'ambiguïté sous-jacente. Un bloc `try/catch` vide avale silencieusement une exception. Aucune de ces pratiques n'est acceptée dans le workflow de revue. Si une correction introduit un contournement, la pull request est renvoyée avec une demande d'investigation de la cause racine.

Cette exigence est documentée dans les critères de revue de pull request et s'applique indépendamment de la criticité du bogue. Elle est l'un des indicateurs de maturité d'ingénierie que le jury de titre évalue lors de la soutenance.

7.4 Mode démo

Le mode démo (`DEMO_MODE=true`) est le filet de sécurité de la démonstration de soutenance et de la phase de recette en préproduction. Quand cette variable est active, l'ensemble des appels aux APIs externes (Transitous, OSRM, OpenWeatherMap, GBFS Bicloo) est remplacé par des réponses JSON statiques préchargées dans `demo-data/`.

Les fichiers de démo couvrent deux scénarios de mobilité représentatifs du réseau nantais : un trajet par temps ensoleillé, où le scoring favorise les modes actifs (vélo, marche), et un trajet par temps pluvieux, où le scoring favorise les transports en commun couverts. Ces deux scénarios permettent de démontrer en conditions maîtrisées le comportement différentiel du moteur de scoring selon la météo, sans aucune dépendance réseau.

Ce dispositif répond à deux besoins simultanés. Il isole les tests d'intégration de toute indisponibilité d'une API tierce pendant le développement. Il garantit que la démonstration reste fonctionnelle même si Transitous ou OSRM, hébergés sur des infrastructures publiques tierces, sont temporairement indisponibles le jour de la soutenance.

Le mode démo n'est pas un prototype simplifié : il exécute exactement les mêmes chemins de code que le mode production. La seule différence est la source des données. Ce qui est démontré en mode démo est fonctionnellement identique à ce qui serait déployé en production avec de vraies APIs, et les tests d'intégration exécutés avec `DEMO_MODE=true` valident par extension le comportement du code de production.

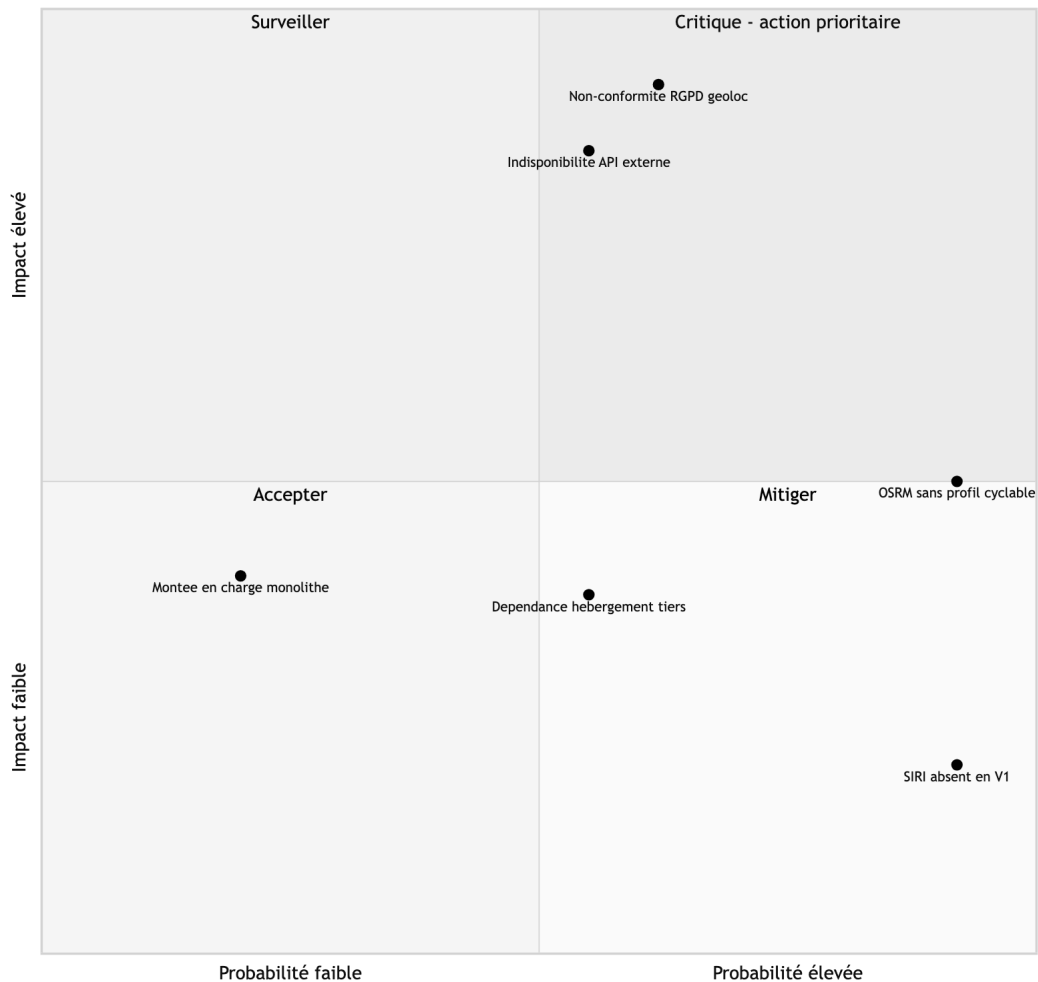
8. Risques, perspectives et conclusion

8.1 Registre de risques

Six risques ont été identifiés et évalués selon deux axes : probabilité d'occurrence sur la durée du projet et impact sur la livraison du MVP. La valeur "Avérée" désigne un risque dont la réalisation est certaine (contrainte connue et documentée), et non une probabilité estimée.

Risque	Probabilité	Impact	Mitigation
Indisponibilité d'une API externe (Transitous ou OSRM)	Moyenne	Élevé	Mode démo (<code>DEMO_MODE=true</code>), timeout individuel de 5 s par provider, fallback JSON statiques
OSRM public limité au profil routier, sans profil cyclable	Avérée	Moyen	Distance géométrique OSRM + durée calculée par vitesse constante par mode (15 km/h vélo, 5 km/h marche, 20 km/h scooter)
Données temps réel SIRI absentes en V1	Avérée	Faible	Horaires théoriques GTFS via Transitous ; intégration SIRI-Lite documentée et réservée à la V2
Montée en charge dépassant la capacité du monolithe	Faible	Moyen	Architecture modulaire : les modules de routage et de gamification sont découplés et extractibles en services indépendants si le besoin émerge
Non-conformité RGPD sur la géolocalisation	Moyenne	Élevé	Consentement explicite avant toute activation, minimisation des données transmises, endpoint <code>DELETE /api/users/me</code> actif, rétention bornée à 12 mois
Dépendance aux tiers gratuits d'hébergement	Moyenne	Moyen	Architecture conteneurisée, aucun appel à des APIs propriétaires de plateforme : migration vers un hébergement souverain sans réécriture applicative

Matrice de risques



Matrice de risques

La matrice reprend les six risques du tableau de la section H1 sur les mêmes axes Probabilité/Impact, répartis en quatre zones d'action.

Critique, action prioritaire (probabilité élevée, impact élevé). L'indisponibilité d'une API externe et la non-conformité RGPD sur la géolocalisation sont les deux risques qui cumulent une probabilité significative et un impact élevé sur la livraison du MVP. Ce sont les seuls risques du registre pour lesquels une mitigation active est en place dès le développement (mode démo, timeout par provider, consentement explicite), et non reportée en réaction.

Mitiger (probabilité élevée, impact modéré à faible). La limitation du profil OSRM public au routage automobile et l'absence de données SIRI temps réel sont des contraintes avérées et documentées avant même le démarrage du développement : leur probabilité est maximale par construction, mais leur impact reste contenu par les approximations et fallbacks déjà en place (vitesses constantes par mode, horaires théoriques GTFS).

Surveiller (probabilité faible, impact modéré). La montée en charge dépassant la capacité du monolithe est peu probable sur la durée d'un MVP à faible base d'utilisateurs, mais son impact justifierait une réaction rapide si le besoin de scalabilité émergeait plus tôt que prévu.

Accepter (probabilité et impact modérés). La dépendance aux tiers gratuits d'hébergement (Vercel, Render, Supabase) est un risque structurel de tout prototype académique, dont l'impact est borné par l'architecture conteneurisée qui permet une migration sans réécriture applicative.

Profil vélo OSRM

OSRM (Open Source Routing Machine, moteur de calcul d'itinéraires open source) ne propose, sur son API publique, qu'un profil de routage automobile. Cette limitation est avérée et connue avant le démarrage du développement.

La mitigation retenue dissocie la géométrie du calcul de durée. OSRM fournit la forme réelle du trajet et la distance exacte. La durée est calculée côté serveur à partir d'une vitesse constante propre à chaque mode. Cette approximation est suffisante pour le moteur de scoring multicritères et pour la démonstration sur le réseau nantais. Un garde-fou complémentaire est inclus dans le provider de routage actif : si la station Bicloo la plus proche dépasse 1,5 km de marche, l'itinéraire n'est pas construit, ce qui évite de proposer des segments de marche disproportionnés que le scoring éliminerait de toute façon.

L'auto-hébergement d'une instance OSRM configurée avec un profil cyclable est prévu en V2, où il remplacera cette approximation sans aucun impact sur le reste de la solution.

Indisponibilité des APIs externes

Transitous et OSRM sont des services publics sans engagement de disponibilité contractuelle (SLA, Service Level Agreement). Leur indisponibilité lors d'une recette ou d'une démonstration est un scénario crédible et anticipé.

Le mode démo répond directement à ce risque. Lorsque la variable `DEMO_MODE` est active, tous les appels externes sont remplacés par des réponses JSON statiques couvrant deux scénarios représentatifs du réseau nantais : un trajet par temps ensoleillé et un trajet par temps pluvieux. Le moteur de scoring et l'ensemble du code de traitement restent identiques à la version production. Ce qui est démontré en mode démo est fonctionnellement équivalent à ce qui serait déployé avec de vraies APIs. Ce dispositif est décrit en détail dans la section Qualité, tests et gestion des bogues.

8.2 Perspectives

Les éléments suivants ne sont pas au périmètre du MVP. Ils sont identifiés dès la conception pour garantir que l'architecture retenue ne crée pas d'obstacles à leur intégration future.

Temps réel et données de perturbations (V2). L'interface SIRI-Lite (Service Interface for Real-time Information) de Naolib permettra d'afficher les prochains passages et les perturbations en temps réel. Le pattern Stratégie (TransportProvider, interface d'abstraction des fournisseurs de transport) est conçu pour absorber ce fournisseur additionnel sans modification du module de routage ni du moteur de scoring.

Routage auto-hébergé (V2). L'auto-hébergement d'une instance OSRM avec profil cyclable, combiné à un serveur OTP (OpenTripPlanner, moteur de planification multimodale open source), améliorera la précision des durées en modes actifs et l'accès à des données TC locales plus riches. Le remplacement des providers est transparent pour le reste de la solution.

Réservation et paiement unifié (V2+). La réservation de vélos Bicloo, de places P+R (Park and Ride, stationnement de transit) et l'achat de titres TC depuis la solution nécessitent des contrats partenaires avec Semitan et les opérateurs de mobilité douce. Ce périmètre est reporté pour des raisons commerciales et réglementaires, non techniques.

Extension multi-collectivités (V2+). L'architecture découplée et la couche d'abstraction TransportProvider permettent de déployer la solution sur une autre métropole en remplaçant uniquement les fournisseurs de données. Ce levier de mutualisation entre collectivités constitue un argument de ROI (retour sur investissement) pour Nantes Métropole : le code produit dans le cadre de ce projet est directement réutilisable sur d'autres territoires.

Personnalisation par l'historique (V2+). Une couche d'adaptation des pondérations du moteur multicritères à partir de l'historique de trajets de l'utilisateur est envisageable. Elle est réservée à la V2 pour les raisons d'éco-conception et de conformité RGPD justifiées dans la section Pilotage.

8.3 Conclusion

UrbanFlow propose à Nantes Métropole une solution construite sur des choix d'ingénierie assumés : une PWA (Progressive Web App) installable sans store, un moteur d'optimisation multicritères déterministe et auditable, une architecture ouverte fondée sur des standards transport reconnus, et un hébergement souverain des données de déplacement citoyennes.

Chaque contrainte du cahier des charges est couverte, tracée et associée à une preuve vérifiable. Les concessions du MVP sont documentées et motivées : absence de données temps réel SIRI, durées en modes actifs calculées par approximation de vitesse, fonctionnalités de réservation reportées en V2. Aucune de ces concessions ne compromet la démonstration du cas d'usage central : le calcul et la comparaison d'itinéraires multimodaux entre deux points du réseau nantais, avec prise en compte des préférences utilisateur, de l'accessibilité PMR et de l'empreinte carbone par segment.

Le ROI proposé à la collectivité est non monétaire. Il se mesure en kilomètres de modes doux substitués à la voiture, en kilogrammes de CO2 évités par usager actif, et en données agrégées d'aide à la décision publique sur les flux de mobilité. Ces métriques sont calculables dès le déploiement en production depuis les données déjà collectées par la solution, sans évolution du modèle de données.

La réutilisabilité de l'architecture sur d'autres collectivités renforce la proposition de valeur au-delà du seul territoire nantais. Nantes Métropole investit dans un bien commun numérique, pas dans un produit à usage unique.

Annexe A : demande initiale du client

Objet : Développement d'une solution de mobilité urbaine intelligente et durable

Bonjour,

Nous représentons une métropole de 500 000 habitants engagée dans la transition vers une mobilité urbaine plus durable et efficace. Face aux défis de congestion, de pollution et de fragmentation des services de transport, nous souhaitons développer une plateforme unifiée qui révolutionne la façon dont nos citoyens se déplacent. Les fonctionnalités que nous envisageons incluent :

- Un planificateur d'itinéraires multimodal intelligent qui combine tous les modes de transport disponibles
- Un système de réservation unifié pour vélos, trottinettes, parkings et places de covoiturage
- Une IA qui optimise les trajets en temps réel selon le trafic, la météo et les préférences utilisateur
- Un système de gamification avec récompenses pour les choix de mobilité durable
- Une plateforme de covoiturage dynamique pour les trajets quotidiens et ponctuels
- Un module de calcul et suivi de l'empreinte carbone des déplacements
- Des alertes et notifications personnalisées (perturbations, alternatives écologiques)
- Un tableau de bord citoyen pour visualiser ses habitudes de mobilité et leur impact
- Une fonctionnalité de signalement collaboratif des problèmes de voirie et transport

Nous aimerions que la plateforme encourage les mobilités douces tout en optimisant l'utilisation de l'infrastructure existante.

Cordialement,

Claire Hénette, UrbanFlow Mobility